EE4077 FUNDAMENTALS OF MACHINE LEARNING

Weeks 2 & 3:

Linear Regression, Overfitting, Bias-Variance Tradeoff

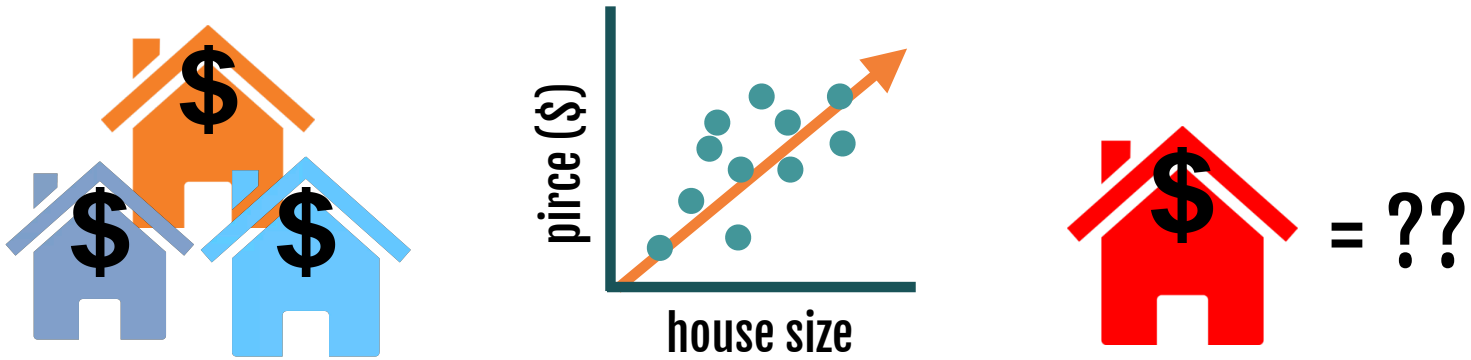
OUTLINE

- Linear regression (algorithm, univariate, multivariate solutions)
- Probabilistic interpretation
- Computational and numerical optimization
- Feature scaling
- Ridge regression
- Non-linear basis functions
- Overfitting and regularization
- Hyperparameter tuning and cross-validation
- Bias-variance trade-off

Task 1: Regression

How much should you sell your house for?





input: houses & features **learn**: $x \rightarrow y$ relationship **predict**: y (*continuous*)

Course Covers: Linear/Ridge Regression, Loss Function, SGD, Feature Scaling, Regularization, Cross Validation

Supervised Learning

Supervised learning

In a supervised learning problem, you have access to input variables (X) and outputs (Y), and the goal is to predict an output given an input

- Examples:
  - **Housing prices (Regression)**: predict the price of a house based on features (size, location, etc)
  - **Cat vs. Dog (Classification)**: predict whether a picture is of a cat or a dog

Regression

Predicting a continuous outcome variable:

- Predicting a company's future stock price using its profit and other financial info
- Predicting annual rainfall based on local flora and fauna
- Predicting distance from a traffic light using LIDAR measurements

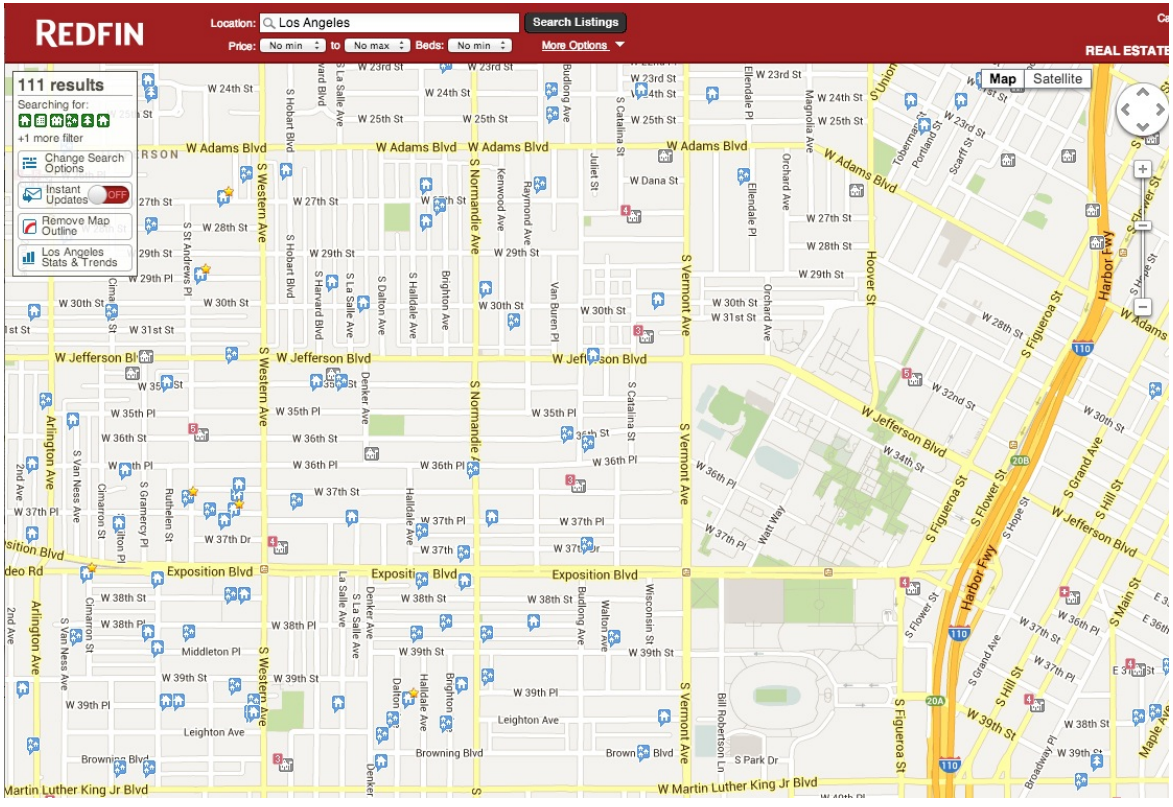
Magnitude of the error matters:

- We can measure 'closeness' of prediction and labels, leading to different ways to evaluate prediction errors.
  - Predicting stock price: better to be off by 1$ than by 20$
  - Predicting distance from a traffic light: better to be off 1 m than by 10 m
- We should choose learning models and algorithms accordingly.

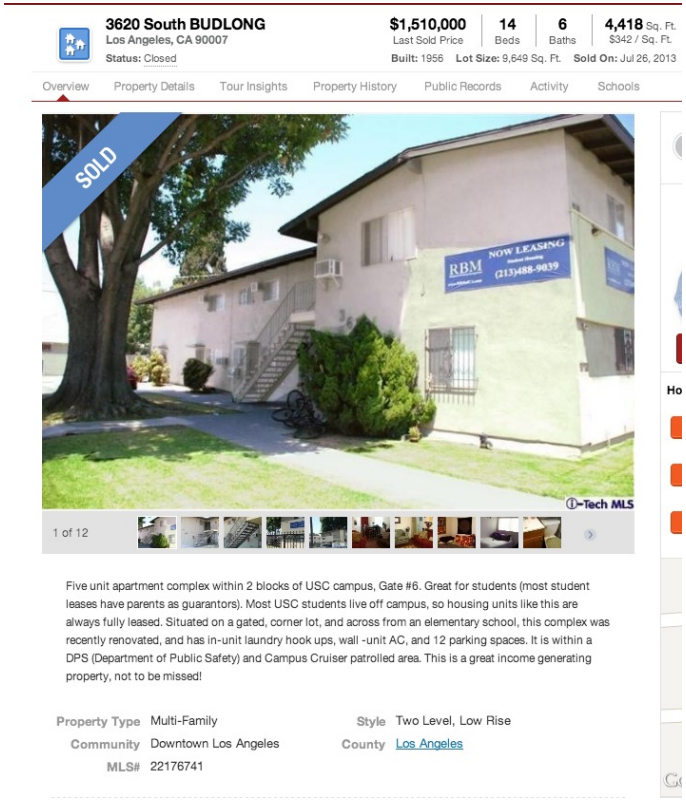
Ex: predicting the sale price of a house

Retrieve historical sales records

(This will be our training data)



Features used to predict



3620 South BUDLONG
Los Angeles, CA 90007
Status: Closed

$1,510,000 Last Sold Price | **14** Beds | **6** Baths | **4,418** Sq. Ft. $342 / Sq. Ft.
Built: 1956 **Lot Size:** 9,649 Sq. Ft. **Sold On:** Jul 26, 2013

Overview | Property Details | Tour Insights | Property History | Public Records | Activity | Schools

1 of 12

Five unit apartment complex within 2 blocks of USC campus, Gate #6. Great for students (most student leases have parents as guarantors). Most USC students live off campus, so housing units like this are always fully leased. Situated on a gated, corner lot, and across from an elementary school, this complex was recently renovated, and has in-unit laundry hook ups, wall -unit AC, and 12 parking spaces. It is within a DPS (Department of Public Safety) and Campus Cruiser patrolled area. This is a great income generating property, not to be missed!

Property Type	Multi-Family	Style	Two Level, Low Rise
Community	Downtown Los Angeles	County	Los Angeles
MLS#	22176741		

Property Details for 3620 South BUDLONG, Los Angeles, CA 90007

Details provided by i-Tech MLS and may not match the public record. Learn More.

Interior Features

Kitchen Information
- Remodeled
- Oven, Range

Laundry Information
- Inside Laundry

Heating & Cooling
- Wall Cooling Unit(s)

Multi-Unit Information

Community Features
- Units in Complex (Total): 5

Multi-Family Information
- # Leased: 5
- # of Buildings: 1
- Owner Pays Water
- Tenant Pays Electricity, Tenant Pays Gas

Unit 1 Information
- # of Beds: 2
- # of Baths: 1
- Unfurnished
- Monthly Rent: $1,700

Unit 2 Information
- # of Beds: 3
- # of Baths: 1
- Unfurnished
- Monthly Rent: $2,250

Unit 3 Information
- Unfurnished

Unit 4 Information
- # of Beds: 3
- # of Baths: 1
- Unfurnished
- Monthly Rent: $2,350

Unit 5 Information
- # of Beds: 3
- # of Baths: 2
- Unfurnished
- Monthly Rent: $2,325

Unit 6 Information
- # of Beds: 3
- # of Baths: 1
- Monthly Rent: $2,250

Property / Lot Details

Property Features
- Automatic Gate, Card/Code Access
- Automatic Gate, Lawn, Sidewalks
- Corner Lot, Near Public Transit

Lot Information
- Lot Size (Sq. Ft.): 9,649
- Lot Size (Acres): 0.2215
- Lot Size Source: Public Records

Property Information
- Updated/Remodeled
- Square Footage Source: Public Records
- Tax Parcel Number: 5040017019

Parking / Garage, Exterior Features, Utilities & Financing

Parking Information
- # of Parking Spaces (Total): 12
- Parking Space
- Gated

Building Information
- Total Floors: 2

Utility Information
- Green Certification Rating: 0.00
- Green Location: Transportation, Walkability
- Green Walk Score: 0
- Green Year Certified: 0

Financial Information
- Capitalization Rate (%): 6.25
- Actual Annual Gross Rent: $128,331
- Gross Rent Multiplier: 11.29

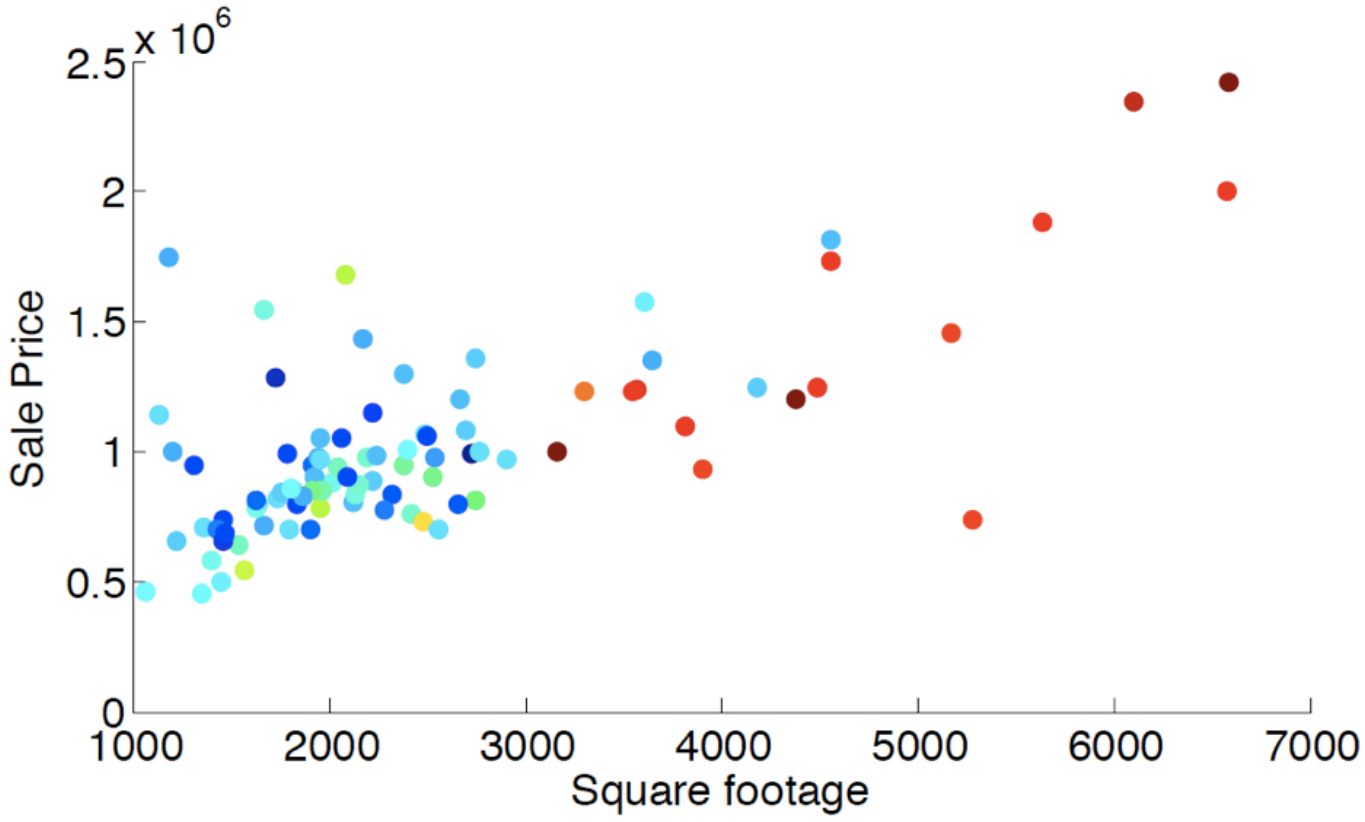
Location Details, Misc. Information & Listing Information

Location Information
- Cross Streets: W 36th Pl

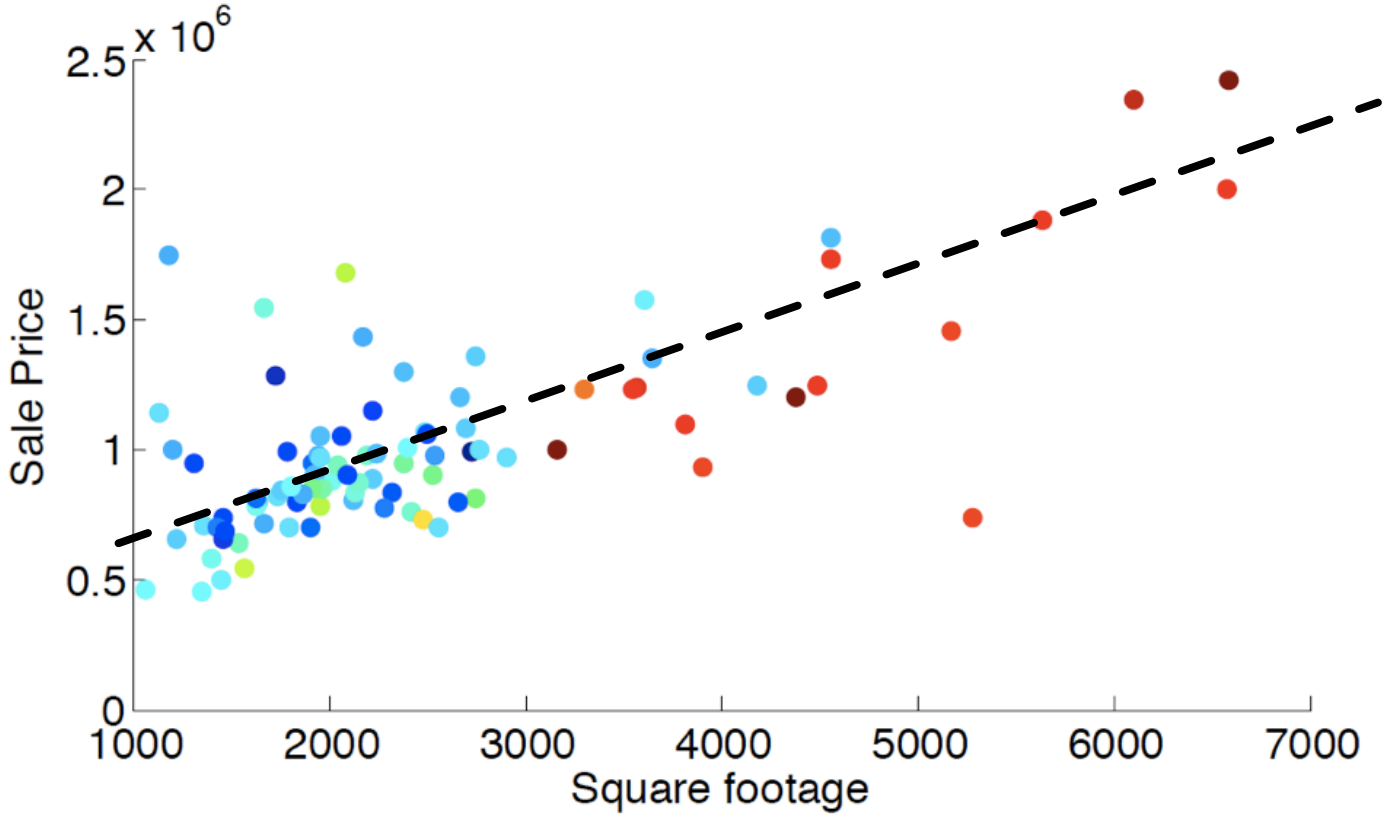
Expense Information
- Operating: $37,664

Listing Information
- Listing Terms: Cash, Cash To Existing Loan
- Buyer Financing: Cash

Correlation between square footage and sale price

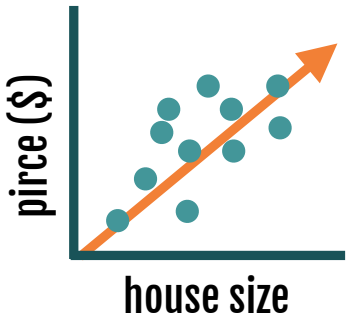


Roughly linear relationship



Sale price $\approx$ price_per_sqft $\times$ square_footage $+$ fixed_expense

Data Can be Compactly Represented by Matrices



- Learn parameters (w_0, w_1) of the orange line $y = w_1 x + w_0$

Sq.ft

$$\text{House 1: } 1000 \times w_1 + w_0 = 200,000$$

$$\text{House 2: } 2000 \times w_1 + w_0 = 350,000$$

- Can represent compactly in matrix notation

$$\begin{bmatrix} 1000 & 1 \\ 2000 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_0 \end{bmatrix} = \begin{bmatrix} 200,000 \\ 350,000 \end{bmatrix}$$

Some Concepts That You Should Know

- Invertibility of Matrices and Computing Inverses
- Vector Norms – L2, Frobenius etc., Inner Products
- Eigenvalues and Eigen-vectors
- Singular Value Decomposition
- Covariance Matrices and Positive Semi-definite-ness

Excellent Resources:

- Essence of Linear Algebra YouTube Series
- Prof. Gilbert Strang's course at MIT

Matrix Inverse

- Let us solve the house-price prediction problem

$$\begin{bmatrix} 1000 & 1 \\ 2000 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_0 \end{bmatrix} = \begin{bmatrix} 200,000 \\ 350,000 \end{bmatrix} \tag{1}$$

$$\begin{bmatrix} w_1 \\ w_0 \end{bmatrix} = \left(\begin{bmatrix} 1000 & 1 \\ 2000 & 1 \end{bmatrix} \right)^{-1} \begin{bmatrix} 200,000 \\ 350,000 \end{bmatrix} \tag{2}$$

$$= \frac{1}{-1000} \begin{bmatrix} 1 & -1 \\ -2000 & 1000 \end{bmatrix} \begin{bmatrix} 200,000 \\ 350,000 \end{bmatrix} \tag{3}$$

$$= \frac{1}{-1000} \begin{bmatrix} 150,000 \\ -5 \times 10^7 \end{bmatrix} \tag{4}$$

$$\begin{bmatrix} w_1 \\ w_0 \end{bmatrix} = \begin{bmatrix} 150 \\ 50,000 \end{bmatrix} \tag{5}$$

You could have data from many houses

- Sale_price =
  price_per_sqft $\times$ square_footage $+$ fixed_expense $+$ unexplainable_stuff
- Want to learn the price_per_sqft and fixed_expense
- Training data: past sales record.

sqft	sale price
2000	800K
2100	907K
1100	312K
5500	2,600K
...	...

Problem: there isn't a $\mathbf{w} = [w_1, w_0]^T$ that will satisfy all equations

Want to predict the best price_per_sqft and fixed_expense

- Sale_price =
  price_per_sqft $\times$ square_footage $+$ fixed_expense $+$ unexplainable_stuff
- Want to learn the price_per_sqft and fixed_expense
- Training data: past sales record.

sqft	sale price	prediction
2000	810K	720K
2100	907K	800K
1100	312K	350K
5500	2,600K	2,600K
...	...	...

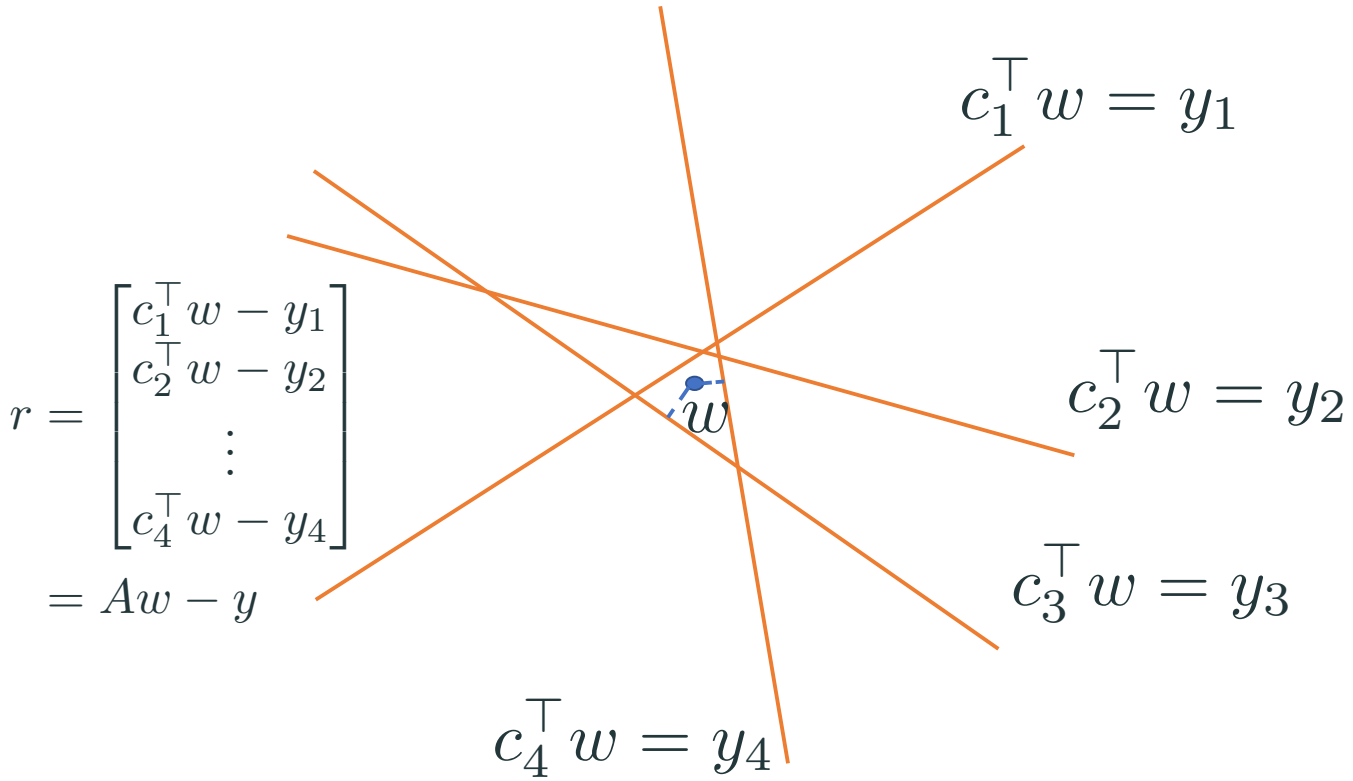
Reduce prediction error

How to measure errors?

- absolute difference: $|\text{prediction} - \text{sale price}|$.
- squared difference: $(\text{prediction} - \text{sale price})^2$ [differentiable!].

sqft	sale price	prediction	abs error	squared error
2000	810K	720K	90K	8100
2100	907K	800K	107K	107^2
1100	312K	350K	38K	38^2
5500	2,600K	2,600K	0	0
...	...			

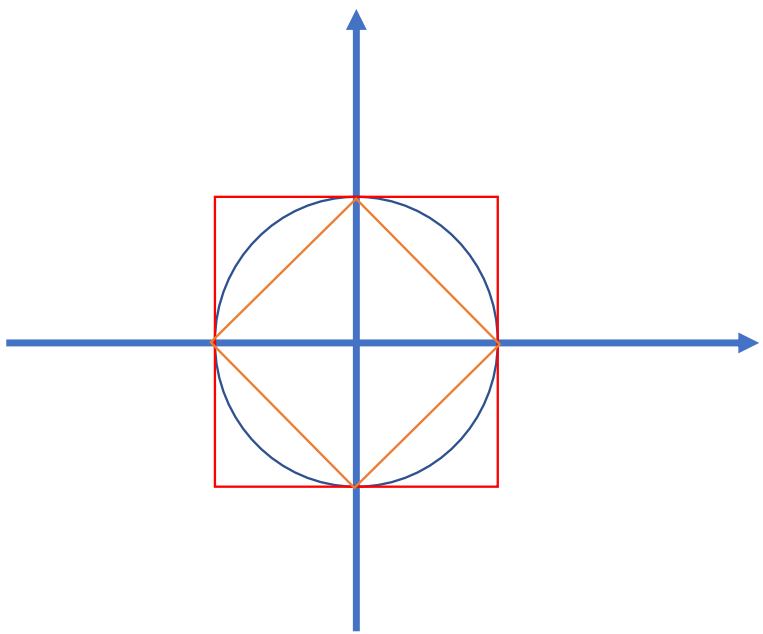
Geometric Illustration: Each house corresponds to one line



- Want to find **w** that minimizes the difference between **Xw**, **y**
- But since this a vector, we need an operator that can map the residual vector $r(\mathbf{w}) = \mathbf{y} - \mathbf{Xw}$ to a scalar

Norms and Loss Functions

- A vector norm is any function $f : \mathbb{R}^n \to \mathbb{R}$ with
  - $f(x) \geq 0$ and $f(x) = 0 \iff x = 0$
  - $f(ax) = |a| f(x)$ for $a \in \mathbb{R}$
  - triangle inequality: $f(x + y) \leq f(x) + f(y)$
- e.g., ℓ_2 norm: $\|x\|_2 = \sqrt{x^\top x} = \sqrt{\sum_{i=1}^n x_i^2}$
- e.g., ℓ_1 norm: $\|x\|_1 = \sum_{i=1}^n |x_i|$
- e.g., ℓ_∞ norm: $\|x\|_\infty = \max |x_i|$



from inside to outside: ℓ_1, ℓ_2, ℓ_∞ norm ball.

Minimize squared errors

Our model:

Sale_price =

price_per_sqft $\times$ square_footage + fixed_expense + unexplainable_stuff

Training data:

sqft	sale price	prediction	error	squared error
2000	810K	720K	90K	8100
2100	907K	800K	107K	107^2
1100	312K	350K	38K	38^2
5500	2,600K	2,600K	0	0
...	...			
Total				$8100 + 107^2 + 38^2 + 0 + \cdots$

Aim:

Adjust price_per_sqft and fixed_expense such that the sum of the squared error is minimized — i.e., the unexplainable_stuff is minimized.

Linear regression

Setup:

- **Input**: $\mathbf{x} \in \mathbb{R}^D$ (covariates, predictors, features, etc)
- **Output**: $y \in \mathbb{R}$ (responses, targets, outcomes, outputs, etc)
- **Model**: $f : \mathbf{x} \to y$, with $f(\mathbf{x}) = w_0 + \sum_{d=1}^{D} w_d x_d = w_0 + \mathbf{w}^\top \mathbf{x}$.
  - $\mathbf{w} = [w_1 \ w_2 \ \cdots \ w_D]^\top$: *weights*, *parameters*, or *parameter vector*
  - w_0 is called *bias*.
  - Sometimes, we also call $\tilde{\mathbf{w}} = [w_0 \ w_1 \ w_2 \ \cdots \ w_D]^\top$ parameters.
- **Training data**: $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1, 2, \ldots, N\}$

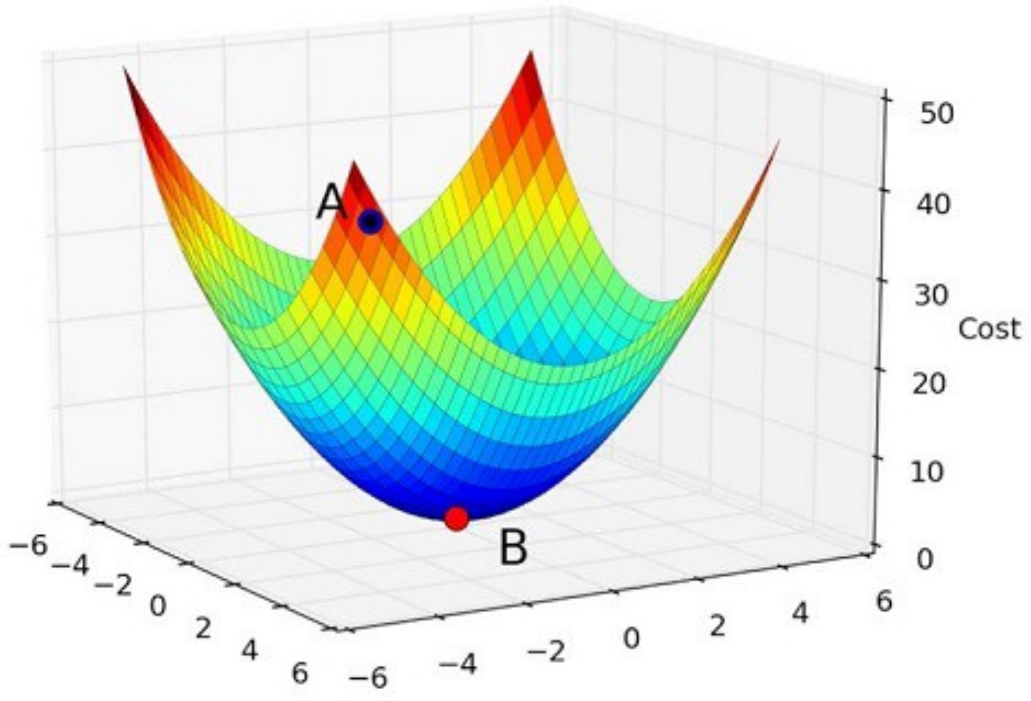
Minimize the Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_{n=1}^{N} [y_n - f(\mathbf{x}_n)]^2 = \sum_{n=1}^{N} [y_n - (w_0 + \sum_{d=1}^{D} w_d x_{nd})]^2$$

A simple case: x is just one-dimensional (D=1)

Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$



What kind of function is this? CONVEX (has a unique global minimum)

A simple case: x is just one-dimensional (D=1)

Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$

Stationary points:

Take derivative with respect to parameters and set it to zero

$$\frac{\partial RSS(\tilde{\mathbf{w}})}{\partial w_0} = 0 \Rightarrow -2 \sum_n [y_n - (w_0 + w_1 x_n)] = 0,$$

$$\frac{\partial RSS(\tilde{\mathbf{w}})}{\partial w_1} = 0 \Rightarrow -2 \sum_n [y_n - (w_0 + w_1 x_n)] x_n = 0.$$

A simple case: x is just one-dimensional (D=1)

$$\frac{\partial RSS(\tilde{\mathbf{w}})}{\partial w_0} = 0 \Rightarrow -2\sum_n [y_n - (w_0 + w_1 x_n)] = 0$$

$$\frac{\partial RSS(\tilde{\mathbf{w}})}{\partial w_1} = 0 \Rightarrow -2\sum_n [y_n - (w_0 + w_1 x_n)] x_n = 0$$

Simplify these expressions to get the "Normal Equations":

$$\sum y_n = N w_0 + w_1 \sum x_n$$

$$\sum x_n y_n = w_0 \sum x_n + w_1 \sum x_n^2$$

Solving the system we obtain the least squares coefficient estimates:

$$w_1 = \frac{\sum (x_n - \bar{x})(y_n - \bar{y})}{\sum (x_i - \bar{x})^2} \qquad \text{and} \qquad w_0 = \bar{y} - w_1 \bar{x}$$

where $\bar{x} = \frac{1}{N}\sum_n x_n$ and $\bar{y} = \frac{1}{N}\sum_n y_n$.

Example

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$

The w_1 and w_0 that minimize this are given by:

$$w_1 = \frac{\sum (x_n - \bar{x})(y_n - \bar{y})}{\sum (x_i - \bar{x})^2} \qquad \text{and} \qquad w_0 = \bar{y} - w_1 \bar{x}$$

where $\bar{x} = \frac{1}{N} \sum_n x_n$ and $\bar{y} = \frac{1}{N} \sum_n y_n$.

Example

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

Residual sum of squares:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - f(\mathbf{x}_n)]^2 = \sum_n [y_n - (w_0 + w_1 x_n)]^2$$

The w_1 and w_0 that minimize this are given by:

$$w_1 \approx 1.6$$

$$w_0 \approx 0.45$$

Least Mean Squares when x is D-dimensional

sqft (1000's)	bedrooms	bathrooms	sale price (100k)
1	2	1	2
2	2	2	3.5
1.5	3	2	3
2.5	4	2.5	4.5

$RSS(\tilde{\mathbf{w}})$ in matrix form:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - (w_0 + \sum_d w_d x_{nd})]^2 = \sum_n [y_n - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n]^2,$$

where we have redefined some variables (by augmenting)

$$\tilde{\mathbf{x}} \leftarrow [1 \; x_1 \; x_2 \; \ldots \; x_D]^\top, \quad \tilde{\mathbf{w}} \leftarrow [w_0 \; w_1 \; w_2 \; \ldots \; w_D]^\top$$

Least Mean Squares when x is D-dimensional

$RSS(\tilde{\mathbf{w}})$ in matrix form:

$$RSS(\tilde{\mathbf{w}}) = \sum_n [y_n - (w_0 + \sum_d w_d x_{nd})]^2 = \sum_n [y_n - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n]^2,$$

where we have redefined some variables (by augmenting)

$$\tilde{\mathbf{x}} \leftarrow [1 \; x_1 \; x_2 \; \ldots \; x_D]^\top, \quad \tilde{\mathbf{w}} \leftarrow [w_0 \; w_1 \; w_2 \; \ldots \; w_D]^\top$$

which leads to

$$\begin{aligned} RSS(\tilde{\mathbf{w}}) &= \sum_n (y_n - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n)(y_n - \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}}) \\ &= \sum_n \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} - 2 y_n \tilde{\mathbf{x}}_n^\top \tilde{\mathbf{w}} + \text{const.} \\ &= \left\{ \tilde{\mathbf{w}}^\top \left(\sum_n \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top \right) \tilde{\mathbf{w}} - 2 \left(\sum_n y_n \tilde{\mathbf{x}}_n^\top \right) \tilde{\mathbf{w}} \right\} + \text{const.} \end{aligned}$$

$RSS(\tilde{\mathbf{w}})$ in new notations

From previous slide:

$$RSS(\tilde{\mathbf{w}}) = \left\{ \tilde{\mathbf{w}}^\top \left(\sum_n \tilde{\mathbf{x}}_n \tilde{\mathbf{x}}_n^\top \right) \tilde{\mathbf{w}} - 2 \left(\sum_n y_n \tilde{\mathbf{x}}_n^\top \right) \tilde{\mathbf{w}} \right\} + \text{const.}$$

Design matrix and target vector:

$$\tilde{\mathbf{X}} = \begin{pmatrix} \tilde{\mathbf{x}}_1^\top \\ \tilde{\mathbf{x}}_2^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Compact expression:

$$RSS(\tilde{\mathbf{w}}) = \|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Example: $RSS(\tilde{\mathbf{w}})$ in compact form

sqft (1000's)	bedrooms	bathrooms	sale price (100k)
1	2	1	2
2	2	2	3.5
1.5	3	2	3
2.5	4	2.5	4.5

Design matrix and target vector:

$$\tilde{\mathbf{X}} = \begin{pmatrix} \tilde{\mathbf{x}}_1^\top \\ \tilde{\mathbf{x}}_2^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

. Compact expression:

$$RSS(\tilde{\mathbf{w}}) = \|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Example: $RSS(\tilde{\mathbf{w}})$ in compact form

sqft (1000's)	bedrooms	bathrooms	sale price (100k)
1	2	1	2
2	2	2	3.5
1.5	3	2	3
2.5	4	2.5	4.5

Design matrix and target vector:

$$\tilde{\mathbf{X}} = \begin{pmatrix} \tilde{\mathbf{x}}_1^\top \\ \tilde{\mathbf{x}}_2^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{pmatrix} = \begin{bmatrix} 1 & 1 & 2 & 1 \\ 1 & 2 & 2 & 2 \\ 1 & 1.5 & 3 & 2 \\ 1 & 2.5 & 4 & 2.5 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 3.5 \\ 3 \\ 4.5 \end{bmatrix}$$

. Compact expression:

$$RSS(\tilde{\mathbf{w}}) = \|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \left\{ \tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - 2 \left(\tilde{\mathbf{X}}^\top \mathbf{y} \right)^\top \tilde{\mathbf{w}} \right\} + \text{const}$$

Solution in matrix form

Compact expression

$$RSS(\tilde{\mathbf{w}}) = ||\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}||_2^2 = \left\{\tilde{\mathbf{w}}^\top \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\tilde{\mathbf{w}} - 2\left(\tilde{\mathbf{X}}^\top \mathbf{y}\right)^\top \tilde{\mathbf{w}}\right\} + \text{const}$$

Gradients of Linear and Quadratic Functions

- $\nabla_{\mathbf{x}}(\mathbf{b}^\top \mathbf{x}) = \mathbf{b}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = 2\mathbf{A}\mathbf{x}$ (symmetric $\mathbf{A}$)

Normal equation

$$\nabla_{\tilde{\mathbf{w}}} RSS(\tilde{\mathbf{w}}) = 2\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\tilde{\mathbf{w}} - 2\tilde{\mathbf{X}}^\top \mathbf{y} = 0$$

This leads to the least-mean-squares (LMS) solution

$$\tilde{\mathbf{w}}^{LMS} = \left(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$$

Example: $RSS(\tilde{\mathbf{w}})$ in compact form

sqft (1000's)	bedrooms	bathrooms	sale price (100k)
1	2	1	2
2	2	2	3.5
1.5	3	2	3
2.5	4	2.5	4.5

Write the least-mean-squares (LMS) solution

$$\tilde{\mathbf{w}}^{LMS} = \left(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$$

Can use solvers in Matlab, Python etc., to compute this for any given $\tilde{\mathbf{X}}$ and $\mathbf{y}$.

Exercise: $RSS(\tilde{\mathbf{w}})$ in compact form

Using the general least-mean-squares (LMS) solution

$$\tilde{\mathbf{w}}^{LMS} = \left(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$$

recover the uni-variate solution that we had computed earlier:

$$w_1 = \frac{\sum (x_n - \bar{x})(y_n - \bar{y})}{\sum (x_i - \bar{x})^2} \qquad \text{and} \qquad w_0 = \bar{y} - w_1 \bar{x}$$

where $\bar{x} = \frac{1}{N} \sum_n x_n$ and $\bar{y} = \frac{1}{N} \sum_n y_n$.

Exercise: $RSS(\tilde{\mathbf{w}})$ in compact form

For the 1-D case, the least-mean-squares solution is

$$\tilde{\mathbf{w}}^{LMS} = \left(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}\right)^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$$

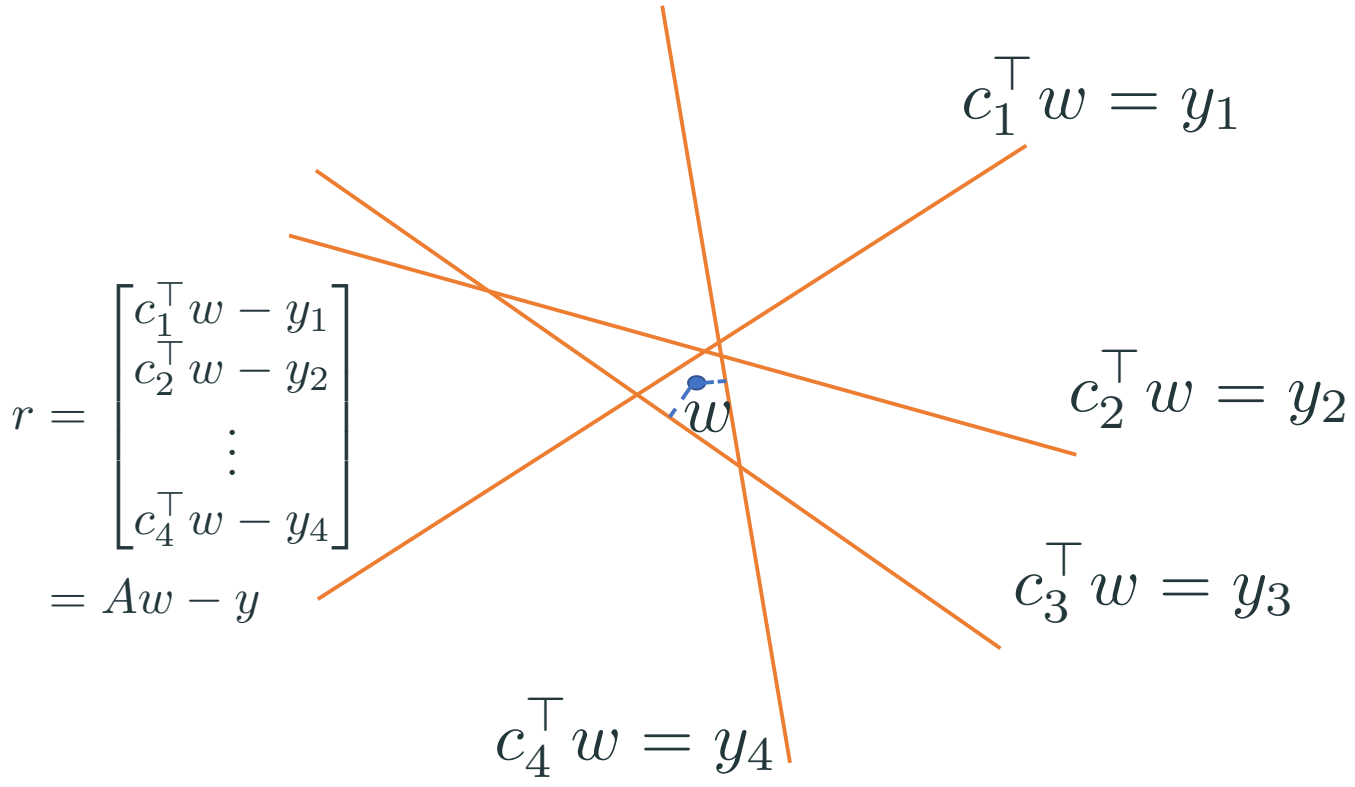
$$= \left(\begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1 & x_2 & \dots & x_N \end{bmatrix} \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & \dots \\ 1 & x_N \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & 1 & \dots & 1 \\ x_1 & x_2 & \dots & x_N \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_N \end{bmatrix}$$

$$= \left(\begin{bmatrix} N & N\bar{x} \\ N\bar{x} & \sum_n x_n^2 \end{bmatrix} \right)^{-1} \begin{bmatrix} \sum_n y_n \\ \sum_n x_n y_n \end{bmatrix}$$

$$\begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \frac{1}{\sum (x_i - \bar{x})^2)} \begin{bmatrix} \bar{y} \sum (x_i - \bar{x})^2 - \bar{x} \sum (x_n - \bar{x})(y_n - \bar{y}) \\ \sum (x_n - \bar{x})(y_n - \bar{y}) \end{bmatrix}$$

where $\bar{x} = \frac{1}{N} \sum_n x_n$ and $\bar{y} = \frac{1}{N} \sum_n y_n$.

Why is minimizing RSS sensible?



- Want to find **w** that minimizes the difference between **Xw**, **y**
- But since this a vector, we need an operator that can map the residual vector $r(\mathbf{w}) = \mathbf{y} - \mathbf{X}\mathbf{w}$ to a scalar
- We take the sum of the squares of the elements of $r(\mathbf{w})$

Why is minimizing RSS sensible?

Probabilistic interpretation

- **Noisy observation model**:

$$Y = w_0 + w_1 X + \eta$$

  where $\eta \sim N(0, \sigma^2)$ is a Gaussian random variable

- Conditional likelihood of one training sample:

$$p(y_n|x_n) = N(w_0 + w_1 x_n, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{[y_n - (w_0 + w_1 x_n)]^2}{2\sigma^2}}$$

Probabilistic interpretation (cont'd)

Log-likelihood of the training data $\mathcal{D}$ (assuming i.i.d):

$$\begin{aligned}
\log P(\mathcal{D}) &= \log \prod_{n=1}^{N} p(y_n|x_n) = \sum_n \log p(y_n|x_n) \\
&= \sum_n \left\{ -\frac{[y_n - (w_0 + w_1 x_n)]^2}{2\sigma^2} - \log \sqrt{2\pi}\sigma \right\} \\
&= -\frac{1}{2\sigma^2} \sum_n [y_n - (w_0 + w_1 x_n)]^2 - \frac{N}{2} \log \sigma^2 - N \log \sqrt{2\pi} \\
&= -\frac{1}{2} \left\{ \frac{1}{\sigma^2} \sum_n [y_n - (w_0 + w_1 x_n)]^2 + N \log \sigma^2 \right\} + \text{const}
\end{aligned}$$

What is the relationship between minimizing RSS and maximizing the log-likelihood?

Maximum likelihood estimation

Estimating σ, w_0 and w_1 can be done in two steps

- Maximize over w_0 and w_1:

$$\max \log P(\mathcal{D}) \Leftrightarrow \min \sum_n [y_n - (w_0 + w_1 x_n)]^2 \leftarrow \text{This is RSS}(\tilde{\mathbf{w}})!$$

- Maximize over $s = \sigma^2$:

$$\frac{\partial \log P(\mathcal{D})}{\partial s} = -\frac{1}{2}\left\{-\frac{1}{s^2}\sum_n [y_n - (w_0 + w_1 x_n)]^2 + N\frac{1}{s}\right\} = 0$$

$$\rightarrow \sigma^{*2} = s^* = \frac{1}{N}\sum_n [y_n - (w_0 + w_1 x_n)]^2$$

How does this probabilistic interpretation help us?

- It gives a solid footing to our intuition: minimizing RSS($\tilde{\mathbf{w}}$) is a sensible thing based on reasonable modeling assumptions.
- Estimating σ^* tells us how much noise there is in our predictions. For example, it allows us to place confidence intervals around our predictions.

Computational complexity of the Least Squares Solution

Bottleneck of computing the solution?

$$\boldsymbol{w} = \left(\boldsymbol{X}^\top \boldsymbol{X}\right)^{-1} \boldsymbol{X}\boldsymbol{y}$$

Matrix multiply of $\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{(\mathrm{D}+1)\times(\mathrm{D}+1)}$
Inverting the matrix $\mathbf{X}^\top \mathbf{X}$

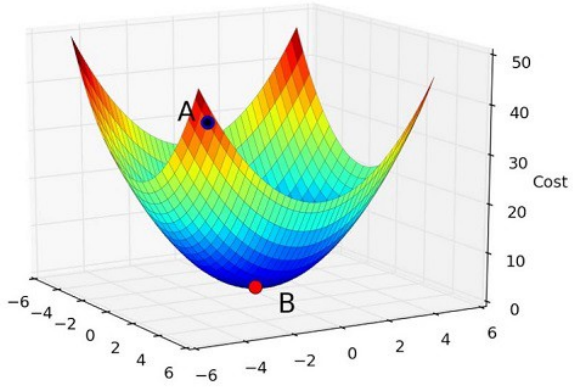
How many operations do we need?

- $O(\mathrm{ND}^2)$ for matrix multiplication
- $O(\mathrm{D}^3)$ (e.g., using Gauss-Jordan elimination) or $O(\mathrm{D}^{2.373})$ (recent theoretical advances) for matrix inversion
- Impractical for very large D or N

Alternative method: Batch Gradient Descent

(Batch) Gradient descent

- Initialize $\mathbf{w}$ to $\boldsymbol{w}^{(0)}$ (e.g., randomly); set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
  1. Compute the gradient $\nabla RSS(\boldsymbol{w}) = \mathbf{X}^\top(\mathbf{X}\boldsymbol{w}^{(t)} - \boldsymbol{y})$
  2. Update the parameters $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w})$
  3. $t \leftarrow t + 1$



What is the complexity of each iteration?

$O(\mathrm{ND})$

Why would this work?

If gradient descent converges, it will converge to the same solution as using matrix inversion.

This is because $RSS(\boldsymbol{w})$ is a convex function in its parameters $\boldsymbol{w}$

Hessian of RSS

$$RSS(\boldsymbol{w}) = \boldsymbol{w}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{w} - 2\left(\mathbf{X}^\top \boldsymbol{y}\right)^\top \boldsymbol{w} + \text{const}$$

$$\Rightarrow \frac{\partial^2 RSS(\boldsymbol{w})}{\partial \boldsymbol{w} \boldsymbol{w}^\top} = 2\mathbf{X}^\top \mathbf{X}$$

$\mathbf{X}^\top \mathbf{X}$ is positive semidefinite, because for any $\boldsymbol{v}$

$$\boldsymbol{v}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{v} = \|\mathbf{X}^\top \boldsymbol{v}\|_2^2 \geq 0$$

Stochastic gradient descent (SGD)

Widrow-Hoff rule: update parameters using one example at a time

- Initialize $\boldsymbol{w}$ to some $\boldsymbol{w}^{(0)}$; set $t = 0$; choose $\eta > 0$
- Loop *until convergence*
  1. random choose a training a sample $\boldsymbol{x}_t$
  2. Compute its contribution to the gradient

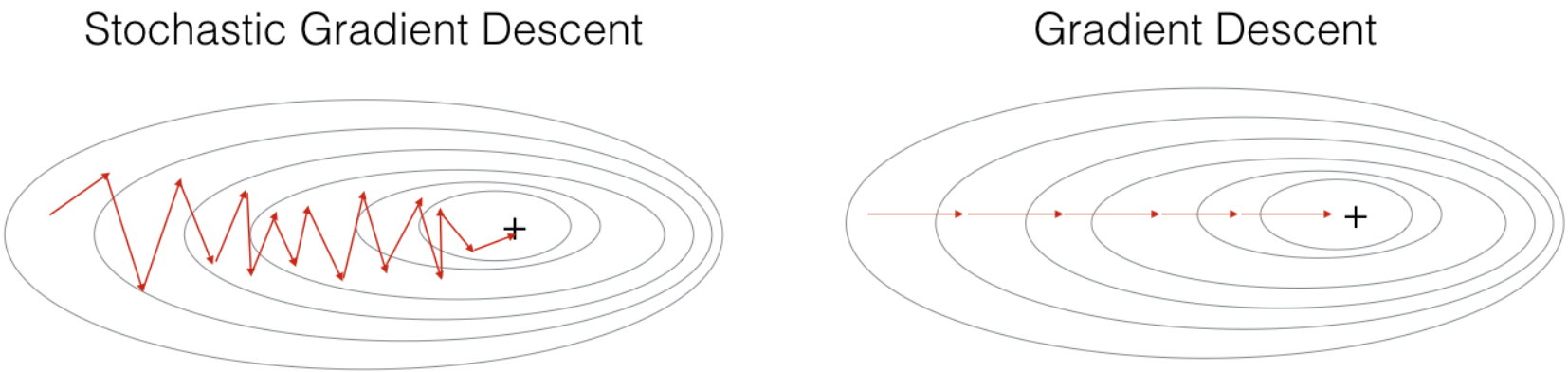
$$\boldsymbol{g}_t = (\boldsymbol{x}_t^\top \boldsymbol{w}^{(t)} - y_t)\mathbf{x}_t$$

  3. Update the parameters
     $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \boldsymbol{g}_t$
  4. $t \leftarrow t + 1$

How does the complexity per iteration compare with gradient descent?

- $O(\mathrm{ND})$ for gradient descent versus $O(\mathrm{D})$ for SGD

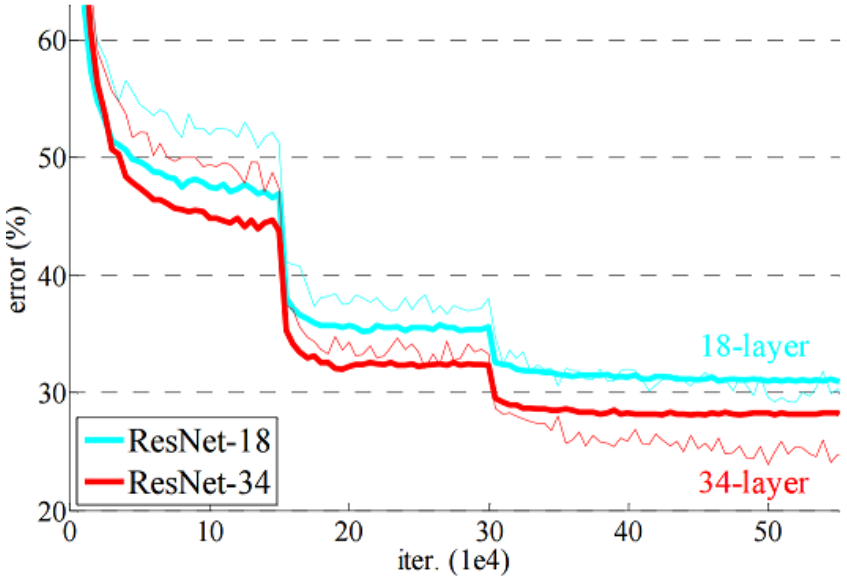
SGD versus Batch GD



- SGD reduces per-iteration complexity from $O(\text{ND})$ to $O(\text{D})$
- But it is noisier and can take longer to converge

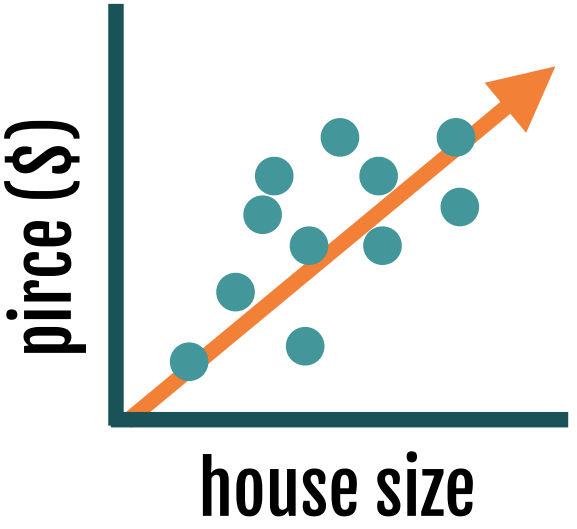
How to Choose Learning Rate η in practice?

- Try $0.0001, 0.001, 0.01, 0.1$ etc. on a validation dataset (more on this later) and choose the one that gives fastest, stable convergence
- Reduce η by a constant factor (eg. 10) when learning saturates so that we can reach closer to the true minimum.
- More advanced learning rate schedules such as AdaGrad, Adam, AdaDelta are used in practice.



Example: Comparing the Three Methods

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5



Example: Least Squares Solution

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

The w_0 and w_1 that minimize this are given by:

$$\mathbf{w}^{LMS} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

$$\begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \left(\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 1.5 & 2.5 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 1.5 \\ 1 & 2.5 \end{bmatrix} \right)^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 1.5 & 2.5 \end{bmatrix} \begin{bmatrix} 2 \\ 3.5 \\ 3 \\ 4.5 \end{bmatrix}$$

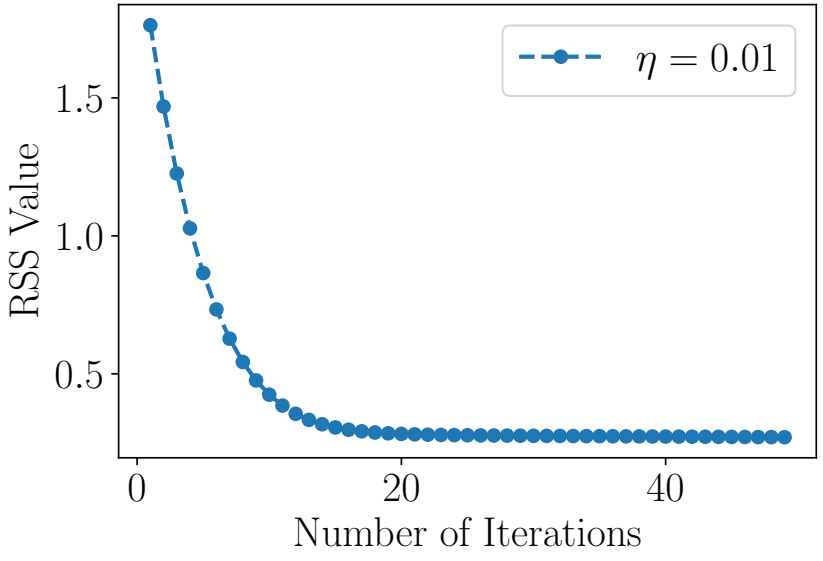
$$\begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} 0.45 \\ 1.6 \end{bmatrix}$$

Minimum RSS is $RSS^* = ||\mathbf{X}\mathbf{w}^{LMS} - \mathbf{y}||_2^2 = 0.2236$

Example: Batch Gradient Descent

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

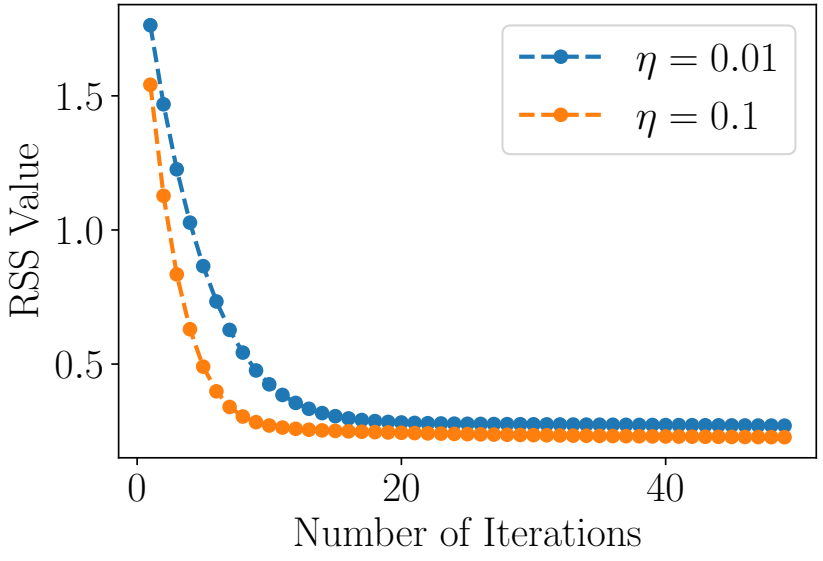
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \mathbf{X}^\top \left(\mathbf{X}\boldsymbol{w}^{(t)} - \boldsymbol{y} \right)$$



Larger η gives faster convergence

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

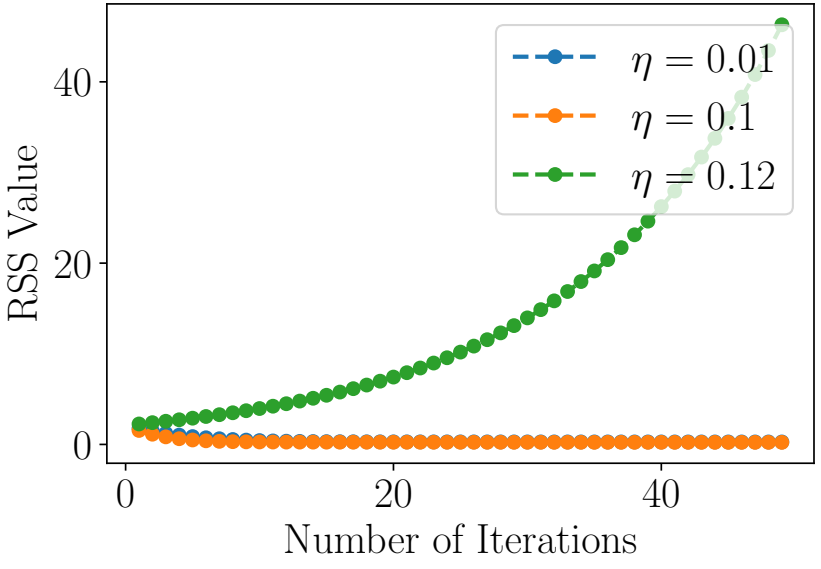
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \mathbf{X}^\top \left(\mathbf{X}\boldsymbol{w}^{(t)} - \boldsymbol{y} \right)$$



But too large η makes GD unstable

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

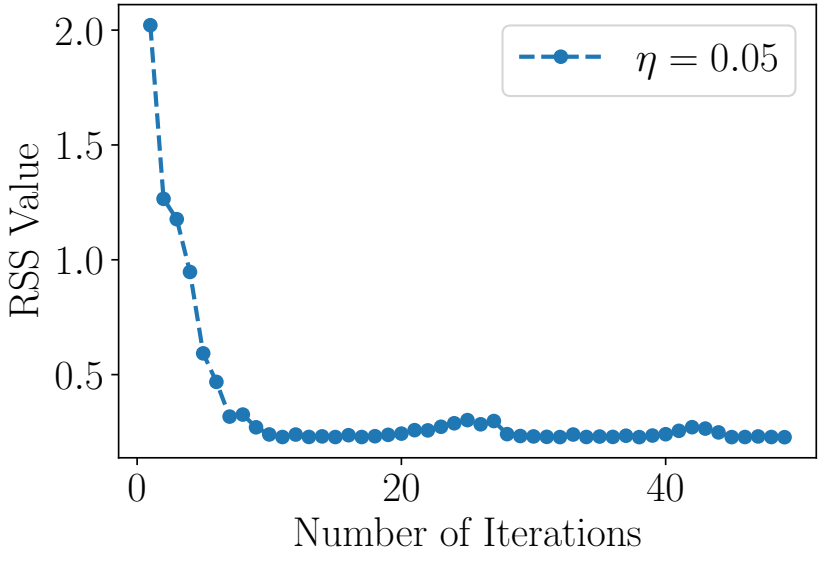
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \mathbf{X}^\top \left(\mathbf{X}\boldsymbol{w}^{(t)} - \boldsymbol{y} \right)$$



Example: Stochastic Gradient Descent

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

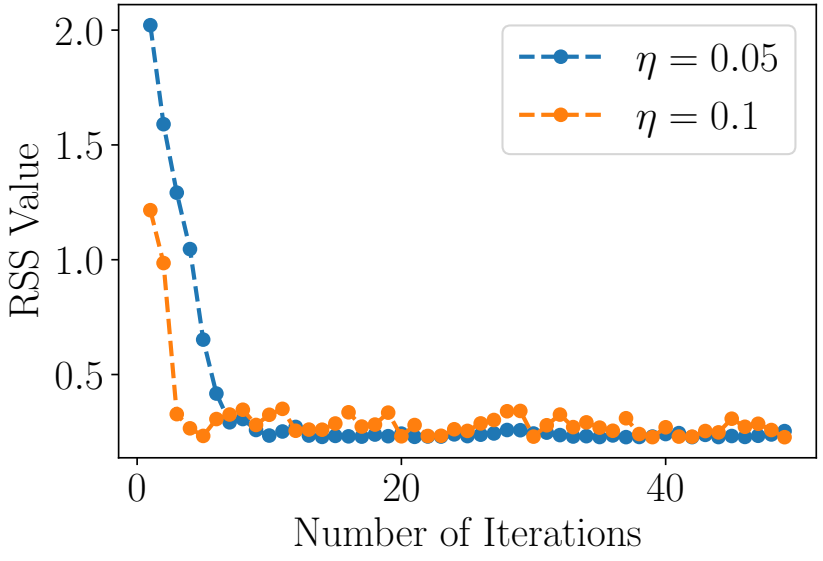
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \left(\mathbf{x}_t^\top \boldsymbol{w}^{(t)} - \boldsymbol{y} \right) \mathbf{x}_t$$



Larger η gives faster convergence

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

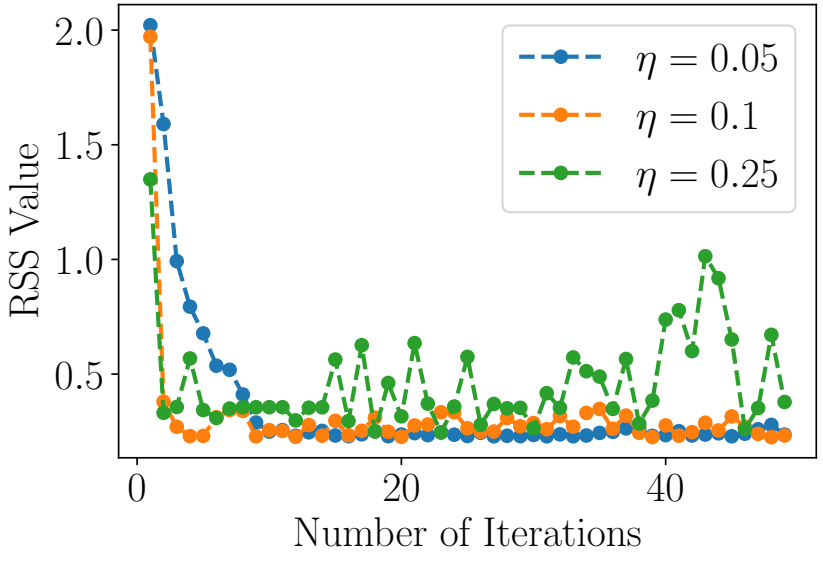
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \left(\mathbf{x}_t^\top \boldsymbol{w}^{(t)} - \boldsymbol{y} \right) \mathbf{x}_t$$



But too large η makes SGD unstable

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

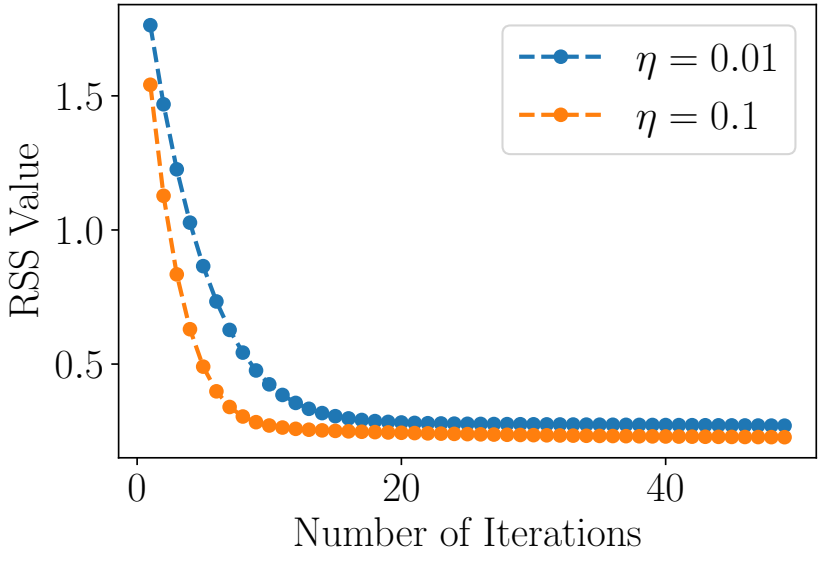
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \left(\mathbf{x}_t^\top \boldsymbol{w}^{(t)} - \boldsymbol{y} \right) \mathbf{x}_t$$



Batch Gradient Descent: Scaled Features

sqft (1000's)	sale price (100k)
1	2
2	3.5
1.5	3
2.5	4.5

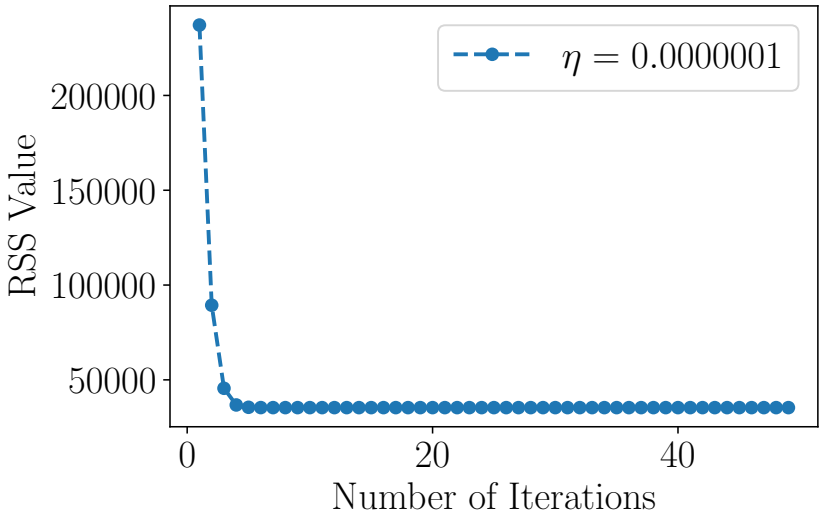
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla RSS(\boldsymbol{w}) = \boldsymbol{w}^{(t)} - \eta \mathbf{X}^\top \left(\mathbf{X}\boldsymbol{w}^{(t)} - \boldsymbol{y} \right)$$



Batch Gradient Descent: Without Feature Scaling

sqft	sale price
1000	200,000
2000	350,000
1500	300,000
2500	450,000

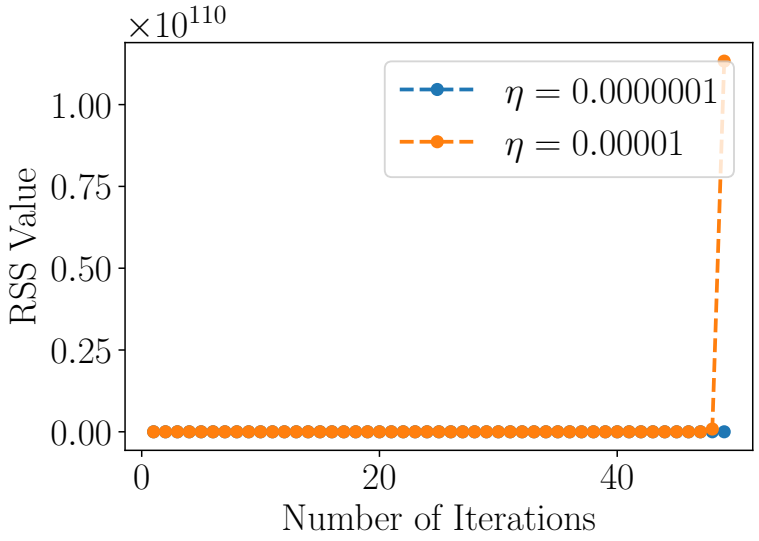
- Least-squares solution is $(w_0^*, w_1^*) = (45000, 160)$
- $\nabla RSS(\boldsymbol{w}^{(t)}) = \mathbf{X}^\top \left(\mathbf{X}\boldsymbol{w}^{(t)} - \boldsymbol{y}\right)$ becomes HUGE, causing instability
- We need a tiny η to compensate, but this leads to slow convergence



Batch Gradient Descent: Without Feature Scaling

sqft	sale price
1000	200,000
2000	350,000
1500	300,000
2500	450,000

- Least-squares solution is $(w_0^*, w_1^*) = (45000, 160)$
- $\nabla RSS(\boldsymbol{w})$ becomes HUGE, causing instability
- We need a tiny η to compensate, but this leads to slow convergence



How to Scale Features?

- **Min-max normalization**

$$x'_d = \frac{x_d - \min_n(x_d)}{\max_n x_d - \min_n x_d}$$

  The min and max are taken over the possible values $x_d^{(1)}, \ldots x_d^{(N)}$ of x_d in the dataset. This will result in all scaled features $0 \leq x_d \leq 1$

- **Mean normalization**

$$x'_d = \frac{x_d - \text{avg}(x_d)}{\max_n x_d - \min_n x_d}$$

  This will result in all scaled features $-1 \leq x_d \leq 1$

Several other methods: eg. dividing by standard deviation (Z-score normalization) Labels $y^{(1)}, \ldots y^{(N)}$ should be similarly re-scaled

What if $X^\top X$ is not invertible?

$$\mathbf{w}^{LMS} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

Why might this happen?

- **Answer 1:** N $<$ D. Not enough data to estimate all parameters. $\mathbf{X}^\top \mathbf{X}$ is not full-rank
- **Answer 2:** Columns of $\boldsymbol{X}$ are not linearly independent, e.g., some features are linear functions of other features. In this case, solution is not unique. Examples:
  - A feature is a re-scaled version of another, for example, having two features correspond to length in meters and feet respectively
  - Same feature is repeated twice – could happen when there are many features
  - A feature has the same value for all data points
  - Sum of two features is equal to a third feature

Example: Matrix $X^\top X$ is not invertible

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

Design matrix and target vector:

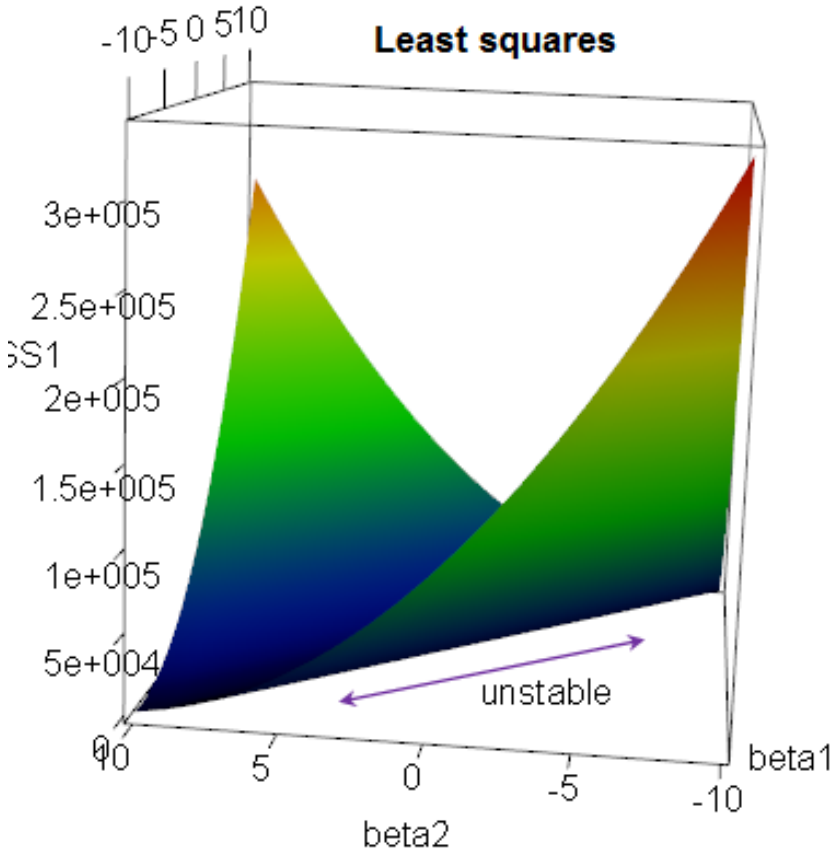
$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 2 \\ 1 & 1.5 & 2 \\ 1 & 2.5 & 2 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 3.5 \\ 3 \\ 4.5 \end{bmatrix}$$

The 'bathrooms' feature is redundant, so we don't need w_2

$$\begin{aligned} y &= w_0 + w_1 x_1 + w_2 x_2 \\ &= w_0 + w_1 x_1 + w_2 \times 2, \quad \text{since } x_2 \text{ is always 2!} \\ &= w_{0,eff} + w_1 x_1, \quad \text{where } w_{0,eff} = (w_0 + 2w_2) \end{aligned}$$

What does the RSS loss function look like?

- When $\boldsymbol{X}^\top \boldsymbol{X}$ is not invertible, the RSS objective function has a ridge, that is, the minimum is a line instead of a single point



In our example, this line is $w_{0,eff} = (w_0 + 2w_2)$

How do you fix this issue?

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

- Manually remove redundant features
- But this can be tedious and non-trivial, especially when a feature is a linear combination of several other features

Need a general way that doesn't require manual feature engineering

SOLUTION: Ridge Regression

Ridge regression

Intuition: what does a non-invertible $\boldsymbol{X}^\top \boldsymbol{X}$ mean?
Consider the SVD of this matrix:

$$\boldsymbol{X}^\top \boldsymbol{X} = \boldsymbol{V} \begin{bmatrix} \lambda_1 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ 0 & \cdots & \cdots & \cdots & 0 \\ 0 & \cdots & \cdots & \lambda_r & 0 \\ 0 & \cdots & \cdots & 0 & 0 \end{bmatrix} \boldsymbol{V}^\top$$

where $\lambda_1 \geq \lambda_2 \geq \cdots \lambda_r > 0$ and $r < \mathrm{D}$. We will have a divide by zero issue when computing $(\boldsymbol{X}^\top \boldsymbol{X})^{-1}$
Fix the problem: ensure all singular values are non-zero:

$$\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I} = \boldsymbol{V} \mathrm{diag}(\lambda_1 + \lambda, \lambda_2 + \lambda, \cdots, \lambda) \boldsymbol{V}^\top$$

where $\lambda > 0$ and $\boldsymbol{I}$ is the identity matrix.

Regularized least square (ridge regression)

Solution

$$\boldsymbol{w} = \left(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

This is equivalent to adding an extra term to $RSS(\boldsymbol{w})$

$$\overbrace{\frac{1}{2}\left\{\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - 2\left(\boldsymbol{X}^\top \boldsymbol{y}\right)^\top \boldsymbol{w}\right\}}^{RSS(\boldsymbol{w})} + \underbrace{\frac{1}{2}\lambda \|w\|_2^2}_{\text{regularization}}$$

Benefits

- Numerically more stable, invertible matrix
- Force **w** to be small
- Prevent overfitting — more on this later

Applying this to our example

sqft (1000's)	bathrooms	sale price (100k)
1	2	2
2	2	3.5
1.5	2	3
2.5	2	4.5

The 'bathrooms' feature is redundant, so we don't need w_2

$$
\begin{aligned}
y &= w_0 + w_1 x_1 + w_2 x_2 \\
&= w_0 + w_1 x_1 + w_2 \times 2, && \text{since } x_2 \text{ is always 2!} \\
&= w_{0,eff} + w_1 x_1, && \text{where } w_{0,eff} = (w_0 + 2w_2) \\
&= 0.45 + 1.6 x_1 && \text{Should get this}
\end{aligned}
$$

Applying this to our example

The 'bathrooms' feature is redundant, so we don't need w_2

$$
\begin{aligned}
y &= w_0 + w_1 x_1 + w_2 x_2 \\
&= w_0 + w_1 x_1 + w_2 \times 2, \quad \text{since } x_2 \text{ is always 2!} \\
&= w_{0,eff} + w_1 x_1, \quad \text{where } w_{0,eff} = (w_0 + 2w_2) \\
&= 0.45 + 1.6 x_1 \quad \text{Should get this}
\end{aligned}
$$

Compute the solution for $\lambda = 0.5$

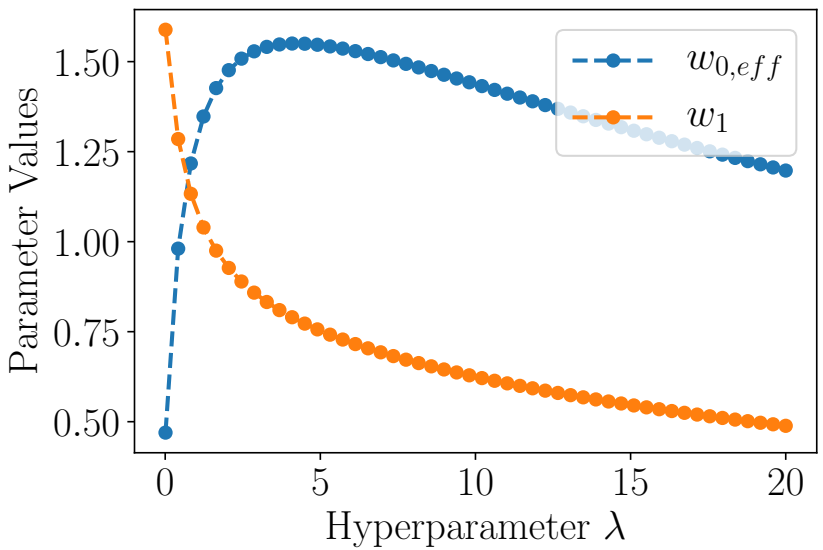
$$
\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \left(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}
$$

$$
\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0.208 \\ 1.247 \\ 0.4166 \end{bmatrix}
$$

How does λ affect the solution?

$$\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \left(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Let us plot $w_o' = w_0 + 2w_2$ and w_1 for different $\lambda \in [0.01, 20]$

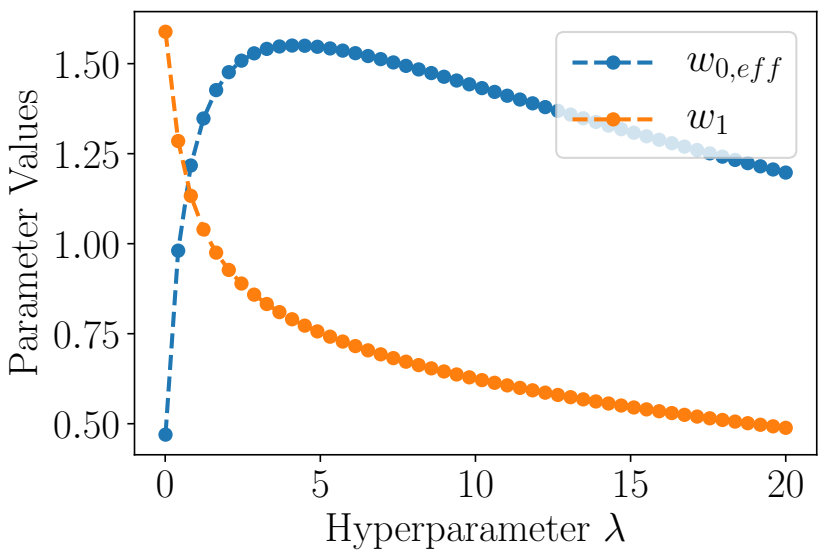


Setting small λ gives almost the least-squares solution, but it can cause numerical instability in the inversion

How to choose λ?

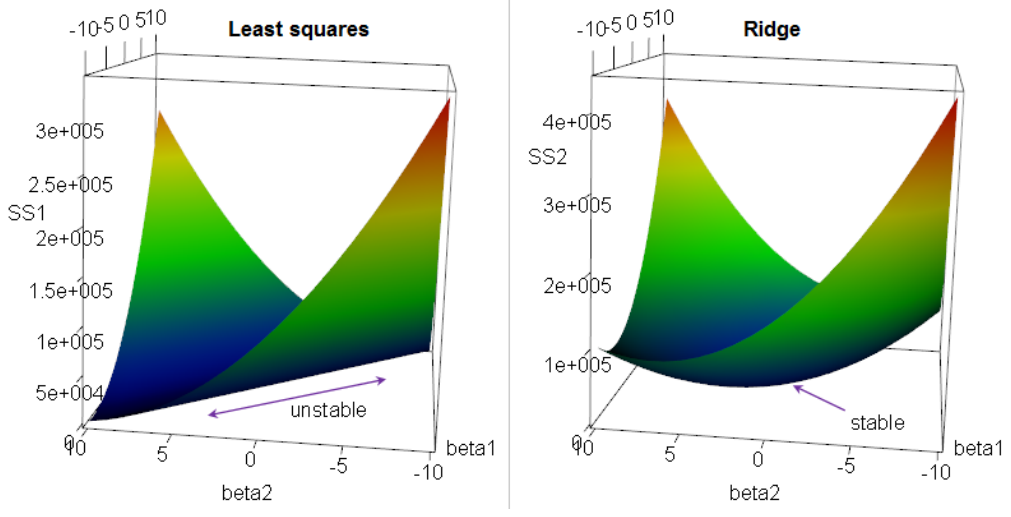
λ is referred as *hyperparameter*

- Associated with the estimation method, not the dataset
- In contrast $\boldsymbol{w}$ is the parameter vector
- Use validation set or cross-validation to find good choice of λ (more on this in the next lecture)



Why is it called Ridge Regression?

- When $\boldsymbol{X}^\top \boldsymbol{X}$ is not invertible, the RSS objective function has a ridge, that is, the minimum is a line instead of a single point
- Adding the regularizer term $\frac{1}{2}\lambda \|w\|_2^2$ yields a unique minimum, thus avoiding instability in matrix inversion



Probabilistic Interpretation of Ridge Regression

Add a term to the objective function.

- Choose the parameters to not just minimize risk, but avoid being too large.

$$\frac{1}{2}\left\{\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - 2\left(\boldsymbol{X}^\top \boldsymbol{y}\right)^\top \boldsymbol{w}\right\} + \frac{1}{2}\lambda\|w\|_2^2$$

Probabilistic interpretation: Place a prior on our weights

- Interpret $\boldsymbol{w}$ as a random variable
- Assume that each w_d is centered around zero
- Use observed data $\mathcal{D}$ to update our prior belief on $\boldsymbol{w}$

Gaussian priors lead to ridge regression.

Review: Probabilistic interpretation of Linear Regression

Linear Regression model: $Y = \boldsymbol{w}^\top \boldsymbol{X} + \eta$

$\eta \sim N(0, \sigma_0^2)$ is a Gaussian random variable and $Y \sim N(\boldsymbol{w}^\top \boldsymbol{X}, \sigma_0^2)$

Frequentist interpretation: We assume that $\boldsymbol{w}$ is fixed.

- The likelihood function maps parameters to probabilities

$$L : \boldsymbol{w}, \sigma_0^2 \mapsto p(\mathcal{D}|\boldsymbol{w}, \sigma_0^2) = p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}, \sigma_0^2) = \prod_n p(y_n|\boldsymbol{x}_n, \boldsymbol{w}, \sigma_0^2)$$

- Maximizing the likelihood with respect to $\boldsymbol{w}$ minimizes the RSS and yields the LMS solution:

$$\boldsymbol{w}^{\mathrm{LMS}} = \boldsymbol{w}^{\mathrm{ML}} = \arg\max_{\boldsymbol{w}} L(\boldsymbol{w}, \sigma_0^2)$$

Probabilistic interpretation of Ridge Regression

Ridge Regression model: $Y = \boldsymbol{w}^\top \boldsymbol{X} + \eta$

- $Y \sim N(\boldsymbol{w}^\top \boldsymbol{X}, \sigma_0^2)$ is a Gaussian random variable (as before)
- $w_d \sim N(0, \sigma^2)$ are i.i.d. Gaussian random variables (unlike before)
- Note that all w_d share the same variance σ^2

- To find $\boldsymbol{w}$ given data $\mathcal{D}$, compute the posterior distribution of $\boldsymbol{w}$:

$$p(\boldsymbol{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})}{p(\mathcal{D})}$$

- Maximum a posterior (MAP) estimate:

$$\boldsymbol{w}^{\text{MAP}} = \arg\max_{\boldsymbol{w}} p(\boldsymbol{w}|\mathcal{D}) = \arg\max_{\boldsymbol{w}} p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})$$

Estimating w

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N$ be i.i.d. with $y|\boldsymbol{w}, \boldsymbol{x} \sim N(\boldsymbol{w}^\top \boldsymbol{x}, \sigma_0^2)$; $w_d \sim N(0, \sigma^2)$.

Joint likelihood of data and parameters (given σ_0, σ):

$$p(\mathcal{D}, \boldsymbol{w}) = p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w}) \quad = \prod_n p(y_n|\boldsymbol{x}_n, \boldsymbol{w}) \prod_d p(w_d)$$

Plugging in the Gaussian PDF, we get:

$$\begin{aligned} \log p(\mathcal{D}, \boldsymbol{w}) &= \sum_n \log p(y_n|\boldsymbol{x}_n, \boldsymbol{w}) + \sum_d \log p(w_d) \\ &= -\frac{\sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2}{2\sigma_0^2} - \sum_d \frac{1}{2\sigma^2} w_d^2 + \text{const} \end{aligned}$$

MAP estimate: $\boldsymbol{w}^{\text{MAP}} = \arg\max_{\boldsymbol{w}} \log p(\mathcal{D}, \boldsymbol{w})$

$$\boldsymbol{w}^{\text{MAP}} = \text{argmin}_{\boldsymbol{w}} \frac{\sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2}{2\sigma_0^2} + \frac{1}{2\sigma^2} \|\boldsymbol{w}\|_2^2$$

Maximum a posterior (MAP) estimate

$$\mathcal{E}(\boldsymbol{w}) = \sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2 + \lambda \|\boldsymbol{w}\|_2^2$$

where $\lambda > 0$ is used to denote σ_0^2/σ^2. This extra term $\|\boldsymbol{w}\|_2^2$ is called regularization/regularizer and controls the magnitude of $\boldsymbol{w}$.

Intuitions

- If $\lambda \to +\infty$, then $\sigma_0^2 \gg \sigma^2$: the variance of noise is far greater than what our prior model can allow for $\boldsymbol{w}$. In this case, our prior model on $\boldsymbol{w}$ will force $\boldsymbol{w}$ to be close to zero. Numerically,

$$\boldsymbol{w}^{\text{MAP}} \to \mathbf{0}$$

- If $\lambda \to 0$, then we trust our data more. Numerically,

$$\boldsymbol{w}^{\text{MAP}} \to \boldsymbol{w}^{\text{LMS}} = \text{argmin} \sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2$$

Is a linear modeling assumption always a good idea?

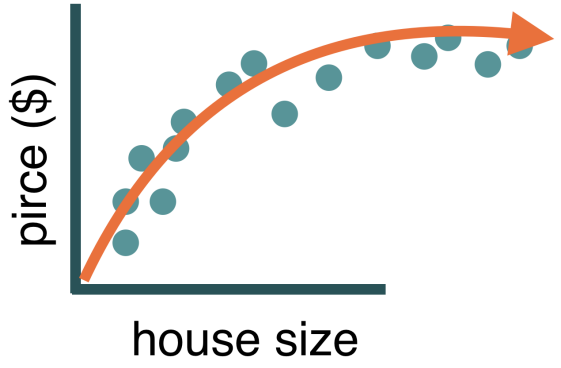


Figure 1: Sale price can saturate as sq.footage increases

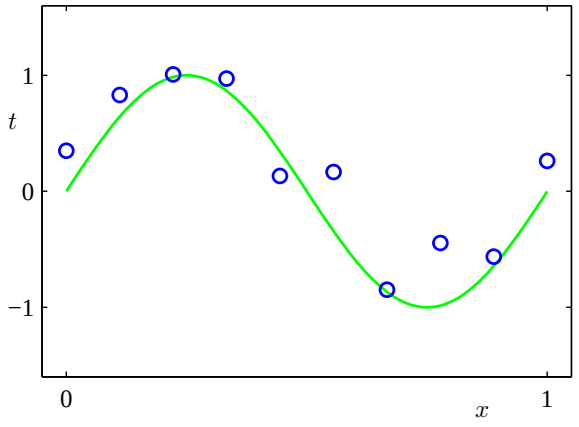


Figure 2: Temperature has cyclic variations over each year

General nonlinear basis functions

We can use a nonlinear mapping to a new feature vector:

$$\boldsymbol{\phi}(\boldsymbol{x}) : \boldsymbol{x} \in \mathbb{R}^D \rightarrow \boldsymbol{z} \in \mathbb{R}^M$$

- M is dimensionality of new features $\boldsymbol{z}$ (or $\boldsymbol{\phi}(\boldsymbol{x})$)
- M could be greater than, less than, or equal to D

We can apply existing learning methods on the transformed data:

- linear methods: prediction is based on $\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x})$
- other methods: nearest neighbors, decision trees, etc

Regression with nonlinear basis

Residual sum of squares

$$\sum_n [\boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x}_n) - y_n]^2$$

where $\boldsymbol{w} \in \mathbb{R}^M$, the same dimensionality as the transformed features $\boldsymbol{\phi}(\boldsymbol{x})$.

The LMS solution can be formulated with the new design matrix

$$\boldsymbol{\Phi} = \begin{pmatrix} \boldsymbol{\phi}(\boldsymbol{x}_1)^\top \\ \boldsymbol{\phi}(\boldsymbol{x}_2)^\top \\ \vdots \\ \boldsymbol{\phi}(\boldsymbol{x}_N)^\top \end{pmatrix} \in \mathbb{R}^{N \times M}, \quad \boldsymbol{w}^{\text{LMS}} = \left(\boldsymbol{\Phi}^\top \boldsymbol{\Phi}\right)^{-1} \boldsymbol{\Phi}^\top \boldsymbol{y}$$

Example: Lot of Flexibility in Designing New Features!

x_1, Area (1k sqft)	x_1^2, Area^2	Price (100k)
1	1	2
2	4	3.5
1.5	2.25	3
2.5	6.25	4.5

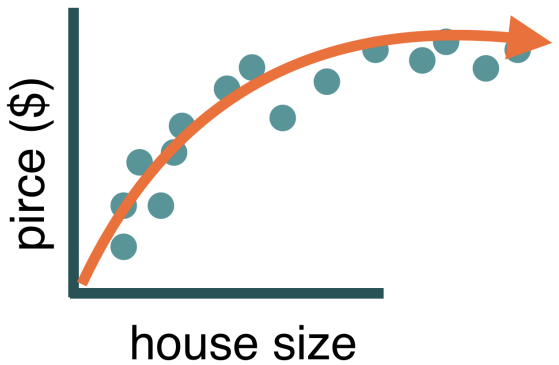


Figure 3: Add x_1^2 as a feature to allow us to fit quadratic, instead of linear functions of the house area x_1

Example: Lot of Flexibility in Designing New Features!

x_1, front (100ft)	x_2 depth (100ft)	$10x_1x_2$, Lot (1k sqft)	Price (100k)
0.5	0.5	2.5	2
0.5	1	5	3.5
0.8	1.5	12	3
1.0	1.5	15	4.5

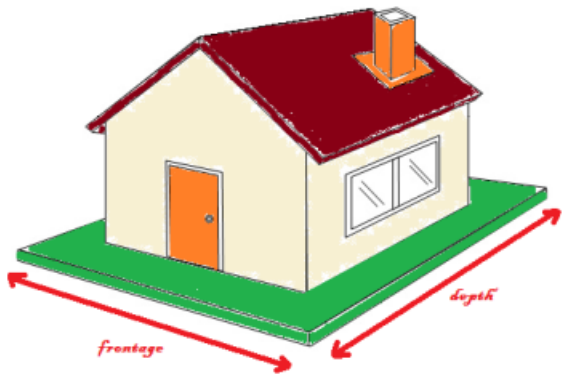


Figure 4: Instead of having frontage and depth as two separate features, it may be better to consider the lot-area, which is equal to frontage$\times$depth

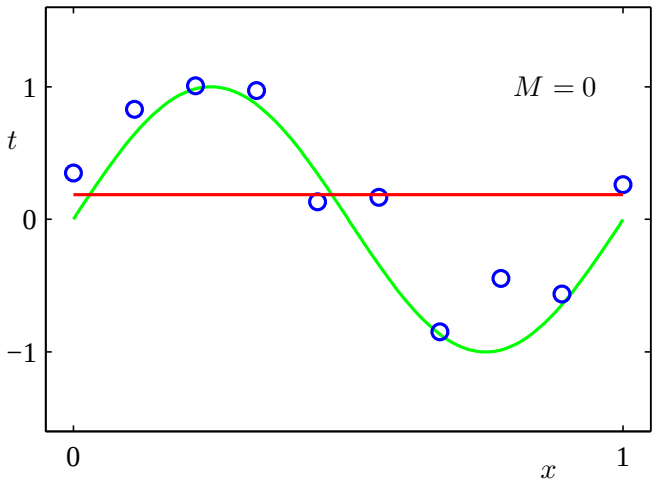
Example with regression

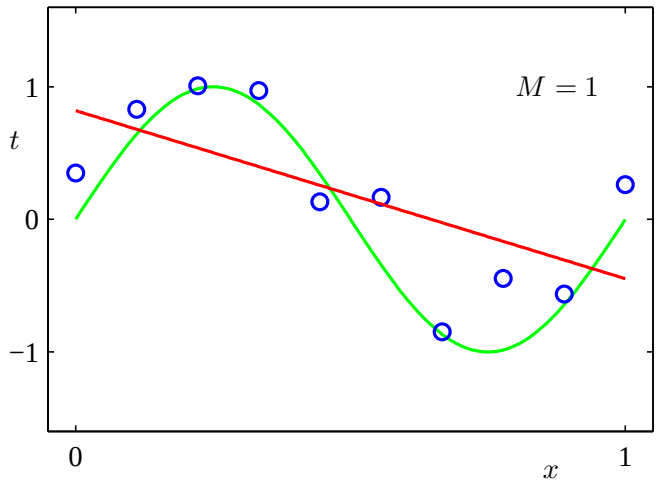
Polynomial basis functions

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{bmatrix} \Rightarrow f(x) = w_0 + \sum_{m=1}^{M} w_m x^m$$

Fitting samples from a sine function:

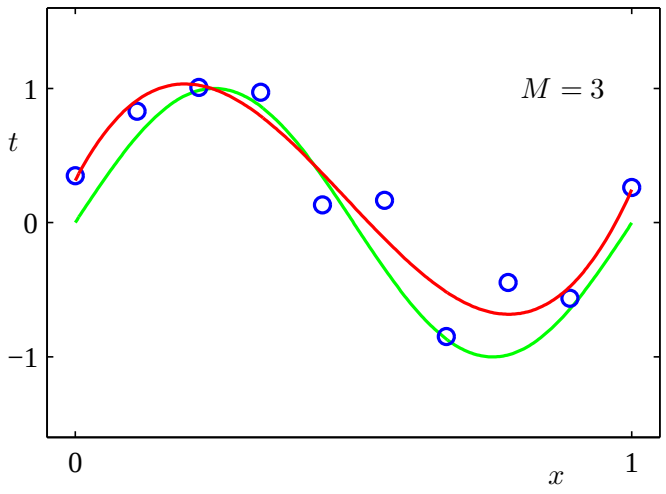
underfitting since $f(x)$ is too simple



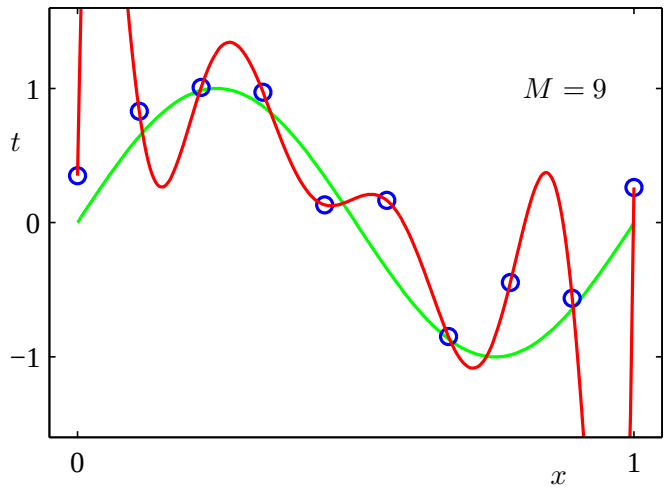


Adding high-order terms

M=3



M=9: overfitting



More complex features lead to better results on the training data, but potentially worse results on new data, e.g., test data!

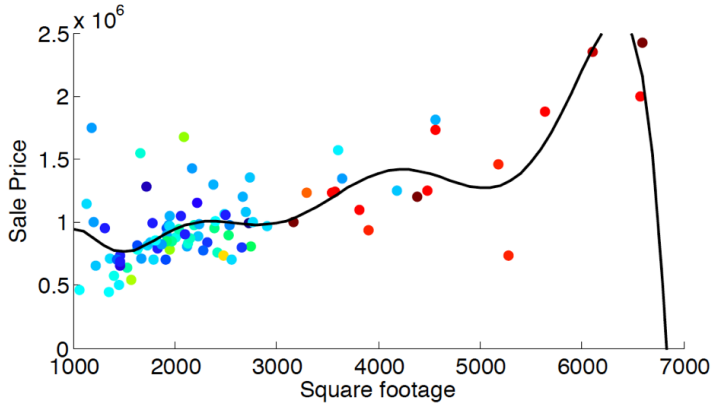
Overfitting

Parameters for higher-order polynomials are very large

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0	0.19	0.82	0.31	0.35
w_1		-1.27	7.99	232.37
w_2			-25.43	-5321.83
w_3			17.37	48568.31
w_4				-231639.30
w_5				640042.26
w_6				-1061800.52
w_7				1042400.18
w_8				-557682.99
w_9				125201.43

Overfitting can be quite disastrous

Fitting the housing price data with large M:



Predicted price goes to zero (and is ultimately negative) if you buy a big enough house!

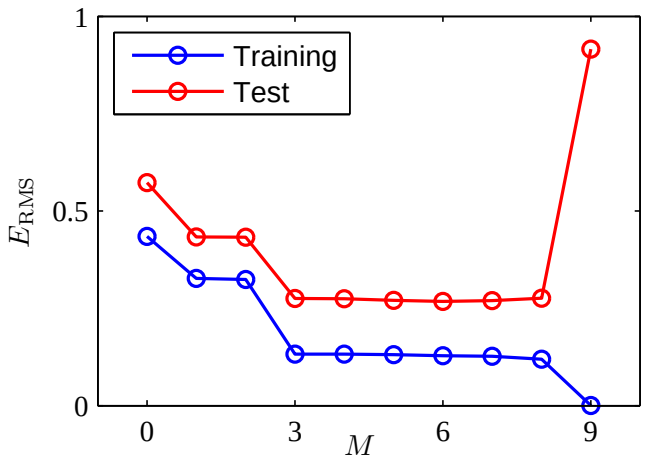
This is called poor generalization/overfitting.

Detecting overfitting

Plot model complexity versus objective function:

- X axis: model complexity, e.g., M
- Y axis: error, e.g., RSS, RMS (square root of RSS), 0-1 loss

Compute the objective on a training and test dataset.

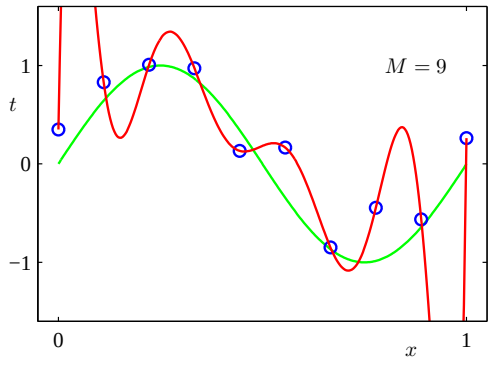


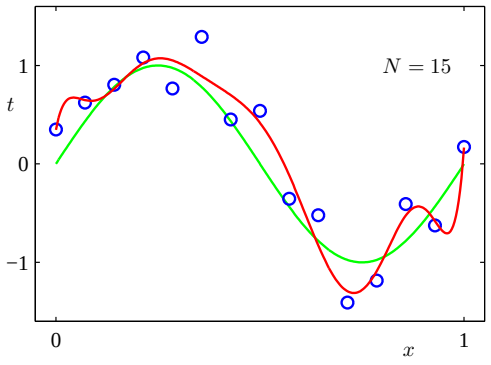
As a model increases in complexity:

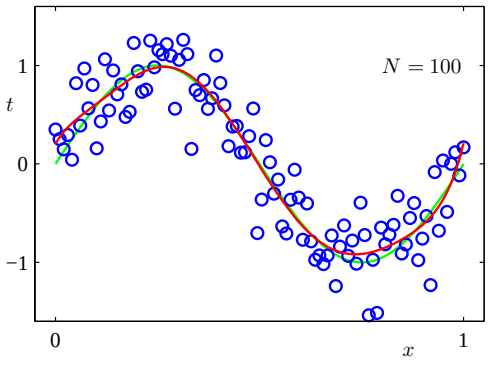
- Training error keeps improving
- Test error may first improve but eventually will deteriorate

Dealing with overfitting: Option 1

Try to use more training data



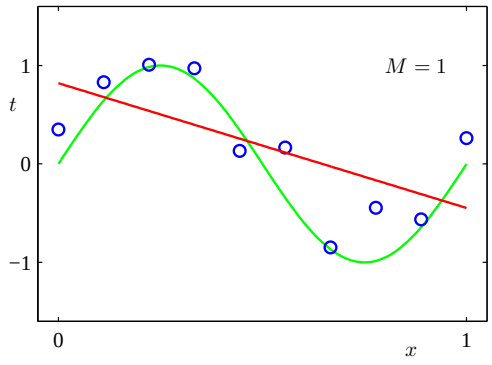


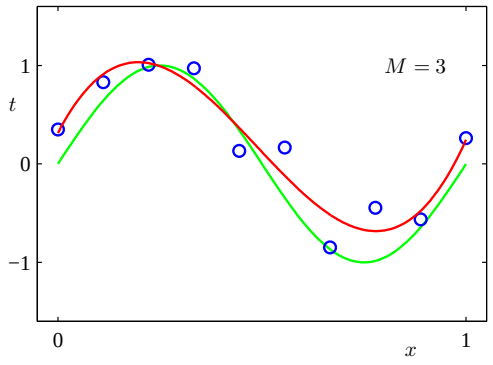


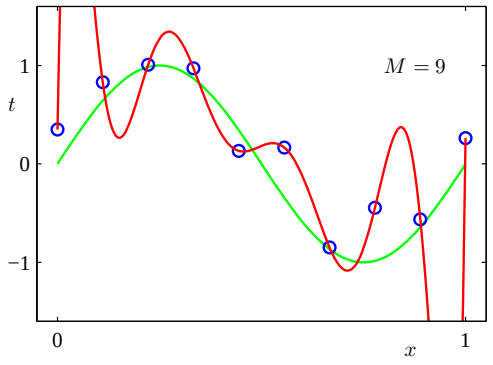
But getting a lot of data can be expensive and time-consuming

Dealing with overfitting: Option 2

Reduce the Number of Features







May not know which and how many features to remove

Dealing with overfitting: Option 3

Regularization Methods: Give preference to 'simpler' models

- How do we define a simple linear regression model — $\boldsymbol{w}^\top \boldsymbol{x}$?
- Intuitively, the weights corresponding to higher order terms should not be "too large"

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0	0.19	0.82	0.31	0.35
w_1		-1.27	7.99	232.37
w_2			-25.43	-5321.83
w_3			17.37	48568.31
w_4				-231639.30
w_5				640042.26
w_6				-1061800.52
w_7				1042400.18
w_8				-557682.99
w_9				125201.43

Regularization methods

Add a term to the objective function.

Choose the parameters to not just minimize risk, but avoid being large.

$$\frac{1}{2}\left\{\boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - 2\left(\boldsymbol{X}^\top \boldsymbol{y}\right)^\top \boldsymbol{w}\right\} + \frac{1}{2}\lambda\|w\|_2^2$$

Ridge regression is just regularized linear regression.

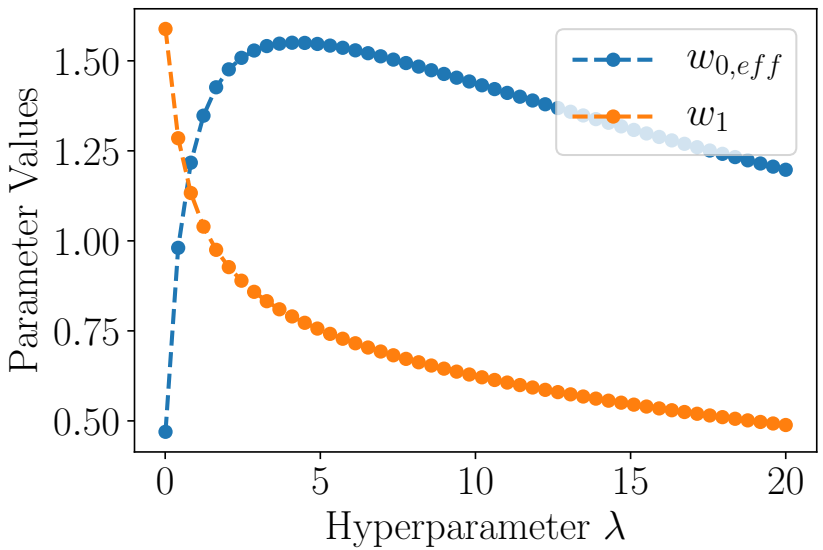
Advantages

- Forces the magnitude of $\boldsymbol{w}$ to be small
- Tries to find a simple model with few parameters
- Generalizes well to new data points

Ridge regression as regularization

$$\begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \left(\boldsymbol{X}^\top \boldsymbol{X} + \lambda \boldsymbol{I} \right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

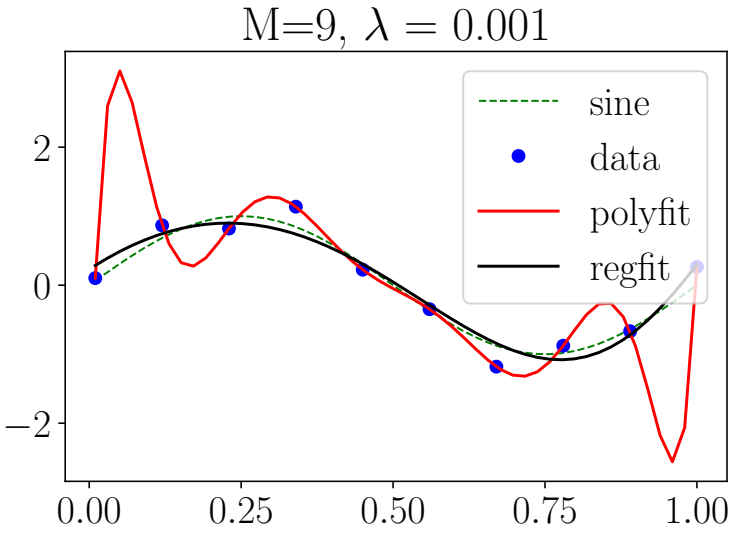
Let us plot $w_o' = w_0 + 2w_2$ and w_1 for different $\lambda \in [0.01, 20]$

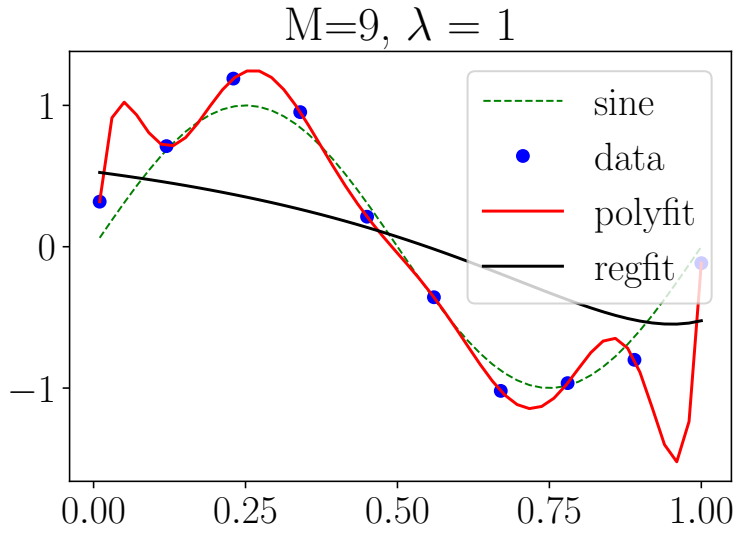


Setting small λ gives almost the least-squares solution, but it can cause numerical instability in the inversion

Example: Effect of regularization

- Regularization makes the higher order w_i's smaller
- Regularized polynomial fit will generalize much better
- As λ increases, the model becomes simpler





Probabilistic interpretation of regularization

Regularized Regression model: $Y = \boldsymbol{w}^\top \boldsymbol{X} + \eta$

- $Y \sim N(\boldsymbol{w}^\top \boldsymbol{X}, \sigma_0^2)$ is a Gaussian random variable (as before)
- $w_d \sim N(0, \sigma^2)$ are i.i.d. Gaussian random variables (unlike before)
- We first choose the weight for each feature d, w_d, from $N(0, \sigma^2)$. Then for each input vector $\boldsymbol{x}_n$, draw y_n from the distribution $N(\boldsymbol{w}^\top \boldsymbol{x}_n, \sigma_0^2)$.

How do we estimate $\boldsymbol{w}$ for this model?

Maximum a posterior (MAP) estimate:

$$\begin{aligned} \boldsymbol{w}^{\text{MAP}} &= \arg\max_{\boldsymbol{w}} p(\boldsymbol{w}|\mathcal{D}) = \arg\max_{\boldsymbol{w}} \frac{p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})}{p(\mathcal{D})} \\ &= \arg\max_{\boldsymbol{w}} p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w}) \end{aligned}$$

Estimating w

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N$ be i.i.d. with $y|\boldsymbol{w}, \boldsymbol{x} \sim N(\boldsymbol{w}^\top \boldsymbol{x}, \sigma_0^2)$; $w_d \sim N(0, \sigma^2)$.

Given σ_0, σ, we choose $\boldsymbol{w}$ so as to maximize:

$$p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w}) = \prod_n p(y_n|\boldsymbol{x}_n, \boldsymbol{w}) \prod_d p(w_d)$$

Now we know $p(w_d) \propto \exp\left(\frac{-w_d^2}{2\sigma^2}\right)$ and $p(y_n|\boldsymbol{x}_n, \boldsymbol{w}) \propto \exp\left(\frac{-(\boldsymbol{w}^T \boldsymbol{x}_n - y_n)^2}{2\sigma_0^2}\right)$:

$$\begin{aligned}\log p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w}) &= \sum_n \log p(y_n|\boldsymbol{x}_n, \boldsymbol{w}) + \sum_d \log p(w_d) \\ &= -\frac{\sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2}{2\sigma_0^2} - \sum_d \frac{1}{2\sigma^2} w_d^2 + \text{const}\end{aligned}$$

MAP estimate: $\boldsymbol{w}^{\text{MAP}} = \arg\max_{\boldsymbol{w}} \log p(\mathcal{D}|\boldsymbol{w})p(\boldsymbol{w})$

$$\boldsymbol{w}^{\text{MAP}} = \text{argmin}_{\boldsymbol{w}} \frac{\sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2}{2\sigma_0^2} + \frac{1}{2\sigma^2} \|\boldsymbol{w}\|_2^2$$

Maximum a posterior (MAP) estimate

$$\mathcal{E}(\boldsymbol{w}) = \sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2 + \lambda \|\boldsymbol{w}\|_2^2$$

where $\lambda > 0$ is used to denote σ_0^2/σ^2. This extra term $\|\boldsymbol{w}\|_2^2$ is called regularization/regularizer and controls the magnitude of $\boldsymbol{w}$.

- If $\lambda \to +\infty$, then $\sigma_0^2 \gg \sigma^2$: the variance of noise is far greater than what our prior model can allow for $\boldsymbol{w}$. In this case, our prior model on $\boldsymbol{w}$ will give a simpler model. Numerically,

$$\boldsymbol{w}^{\mathrm{MAP}} \to \mathbf{0}$$

- If $\lambda \to 0$, then we trust our data more. Numerically,

$$\boldsymbol{w}^{\mathrm{MAP}} \to \boldsymbol{w}^{\mathrm{LMS}} = \operatorname{argmin} \sum_n (\boldsymbol{w}^\top \boldsymbol{x}_n - y_n)^2$$

How should we choose the right amount of regularization?

Can we tune λ on the training dataset?

No: as this will always set λ to zero, i.e., no regularization, defeating our intention of controlling model complexity

λ is thus a hyperparameter. To tune it,

- We can use a validation set or do cross validation.
- Pick the value of λ that yields lowest error on the testing dataset.

Similar idea applies to tuning learning rate η (or any other hyperparameter) as well.

Tuning by using a validation dataset

Training data are used to learn $f(\cdot)$.

N samples/instances: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_N, y_N)\}$

Test data are used to assess the prediction error.

- M samples/instances: $\mathcal{D}^{\text{TEST}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_M, y_M)\}$
- They are used for assessing how well $f(\cdot)$ will do in predicting an unseen $\boldsymbol{x} \notin \mathcal{D}^{\text{TRAIN}}$

Validation data are used to optimize hyperparameter(s).

L samples/instances: $\mathcal{D}^{\text{VAL}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_L, y_L)\}$

Training data, validation and test data should not overlap!

Recipe

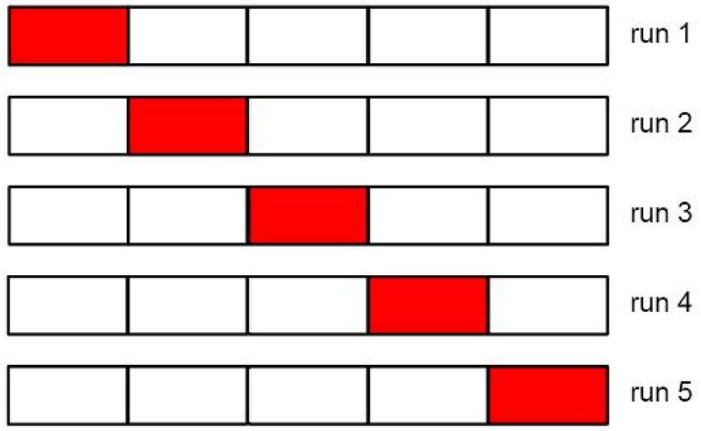
- For each possible value of the hyperparameter (say $\lambda = 1, 3, \cdots, 100$)
  - Train a model using $\mathcal{D}^{\text{TRAIN}}$
  - Evaluate the performance of the model on $\mathcal{D}^{\text{VAL}}$
- Choose the model with the best performance on $\mathcal{D}^{\text{VAL}}$
- Evaluate this model on $\mathcal{D}^{\text{TEST}}$ to get the final prediction error

Cross-validation

What if we do not have validation data?

- We split the training data into S equal parts.
- We use each part in turn as a validation dataset and use the others as a training dataset.
- We choose the hyperparameter such that the model performs the best (based on average, variance, etc.)

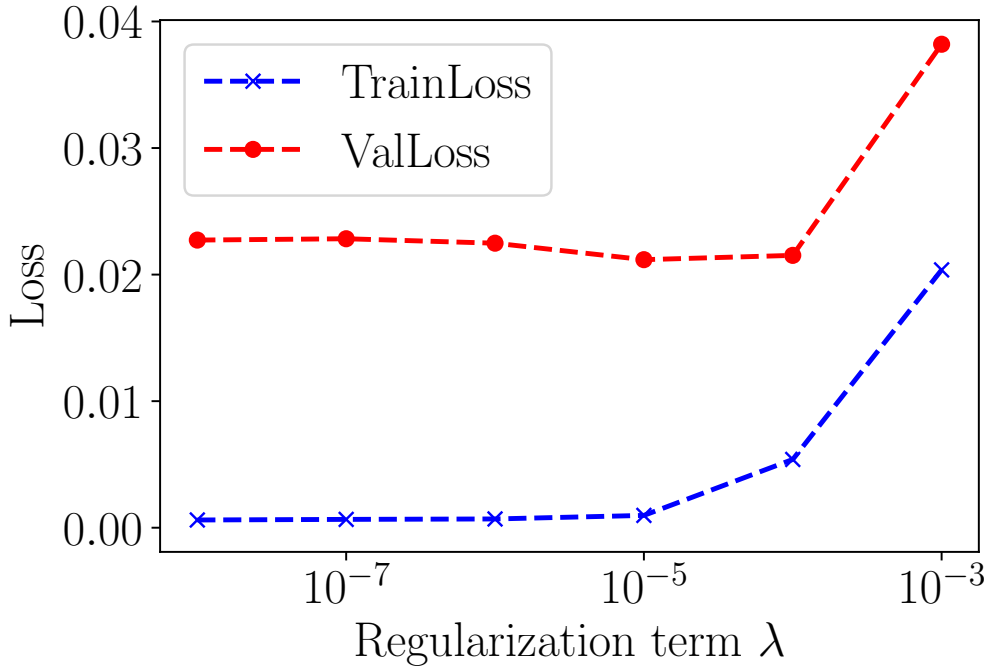
Figure 5: S = 5: 5-fold cross validation



Special case: when S = N, this will be leave-one-out.

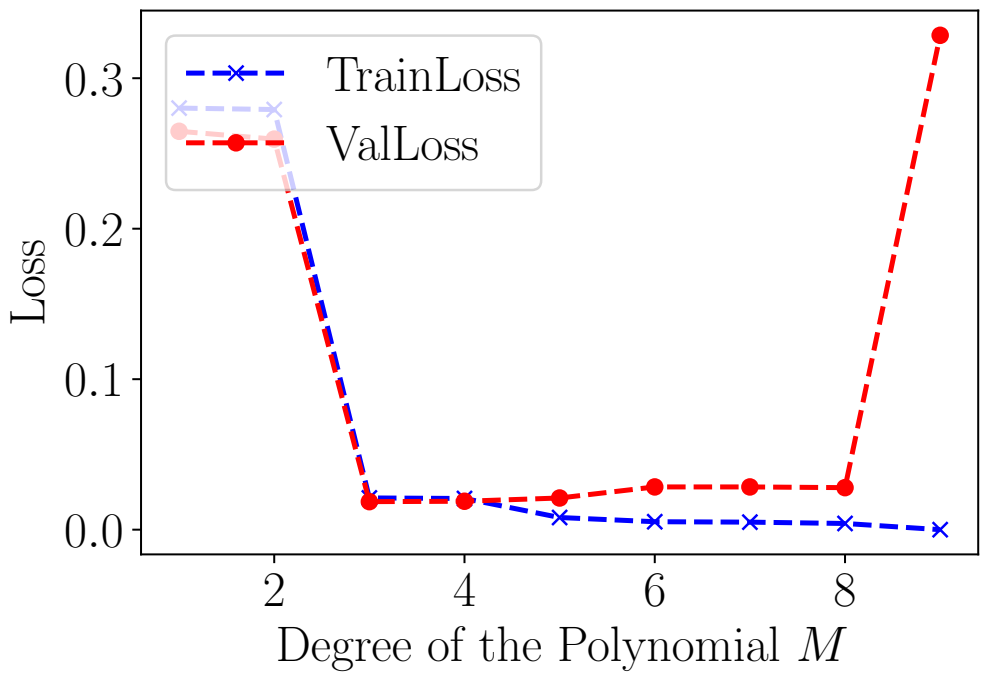
Example: Hyper-parameter Tuning λ

- $\lambda = 10^{-4}$ gives the smallest validation loss
- Strikes a balance between over- and under-fitting



Example: Hyper-parameter Tuning M

- Considering polynomial regression without regularization
- $M = 3$ or $M = 4$ gives the smallest validation loss
- Strikes a balance between over- and under-fitting



Empirical Risk Minimization

Supervised learning

We aim to build a function $h(\boldsymbol{x})$ to predict the true value y associated with $\boldsymbol{x}$. If we make a mistake, we incur a loss

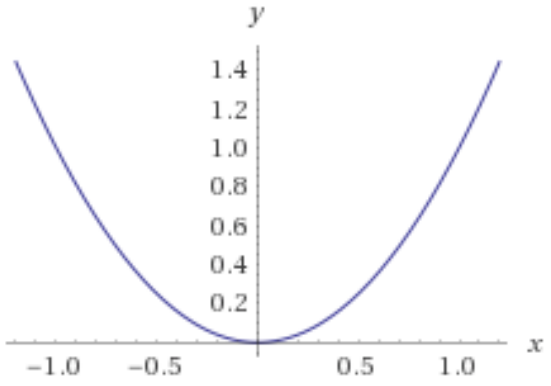
$$\ell(h(\boldsymbol{x}), y)$$

Example:

Quadratic loss function for regression when y is continuous:

$$\ell(h(\boldsymbol{x}), y) = [h(\boldsymbol{x}) - y]^2$$

Ex: when $y = 0$



How good is our predictor?

Risk:

Given the true distribution of data $p(\boldsymbol{x}, y)$, the risk of a given predictor $h(\boldsymbol{x})$ is its expected loss ℓ:

$$R[h(\boldsymbol{x})] = \int_{\boldsymbol{x},y} \ell(h(\boldsymbol{x}), y) p(\boldsymbol{x}, y) d\boldsymbol{x} d\, y$$

However, we cannot compute $R[h(\boldsymbol{x})]$ (we do not know p), so we use the empirical risk, given a training dataset $\mathcal{D}$:

$$R^{\mathrm{EMP}}[h(\boldsymbol{x})] = \frac{1}{N} \sum_{n} \ell(h(\boldsymbol{x}_n), y_n)$$

Intuitively, as $N \to +\infty$,

$$R^{\mathrm{EMP}}[h(\boldsymbol{x})] \to R[h(\boldsymbol{x})]$$

How could this go wrong?

So far, we have been doing empirical risk minimization (ERM)

For linear regression, $h(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$, and we use squared loss ℓ.

$$R^{\text{EMP}}[h(\boldsymbol{x})] = \frac{1}{N} \sum_n \ell(h(\boldsymbol{x}_n), y_n)$$

$$R[h(\boldsymbol{x})] = \int_{\boldsymbol{x},y} \ell(h(\boldsymbol{x}), y) p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

What could go wrong with ERM?

- Limited Function Class: The function $h(\boldsymbol{x})$ is restricted to a limited class (e.g. linear functions), which does not allow us to perfectly fit y, even if we had infinitely many training data points.
- Limited Data: We don't know $p(\boldsymbol{x}, y)$, so we must hope that we have enough training data that the empirical risk approximates the real risk. Otherwise, we will overfit to the training data.

Bias-Variance Trade-off: Intuition

- High Bias: Model is not rich enough to fit the training dataset and achieve low training loss
- High Variance: If the training dataset changes slightly, the model changes a lot
- Regularization helps find a middle ground

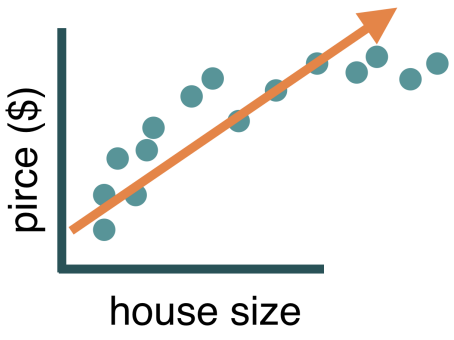


Figure 6: High Bias

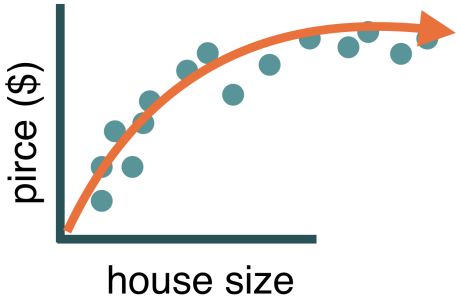


Figure 7: Just Right

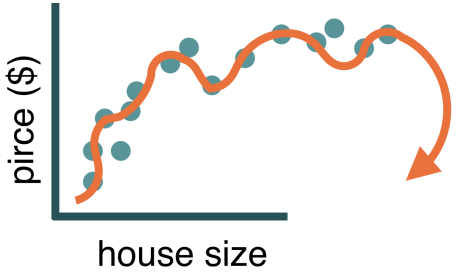


Figure 8: High Variance

Bias/variance tradeoff for regression

Goal: to understand the sources of prediction errors

- $\mathcal{D}$: our training data
- $h_{\mathcal{D}}(\boldsymbol{x})$: our prediction function
  We are using the subscript $\mathcal{D}$ to indicate that the prediction function is learned on the specific set of training data $\mathcal{D}$
- $\ell(h(\boldsymbol{x}), y)$: our square loss function for regression

$$\ell(h_{\mathcal{D}}(\boldsymbol{x}), y) = [h_{\mathcal{D}}(\boldsymbol{x}) - y]^2$$

- Unknown joint distribution $p(\boldsymbol{x}, y)$

The effect of finite training samples

Every training sample $\mathcal{D}$ is a sample from the following joint distribution of all possible training datasets

$$\mathcal{D} \sim P(\mathcal{D}) = \prod_{n=1}^{N} p(\boldsymbol{x}_n, y_n)$$

Thus, the prediction function $h_{\mathcal{D}}(\boldsymbol{x})$ is a random function with respect to this distribution of possible training datasets. So is also its risk

$$R[h_{\mathcal{D}}(\boldsymbol{x})] = \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

We will now evaluate the expected risk $\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\boldsymbol{x})]$: the average risk over the distribution of possible training datasets, $P(\mathcal{D})$.

Bias-Variance Trade-off: Intuition

Error decomposes into 3 terms

$$\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\boldsymbol{x})] = \text{VARIANCE} + \text{BIAS}^2 + \text{NOISE}$$

We will prove this result, and interpret what it means...

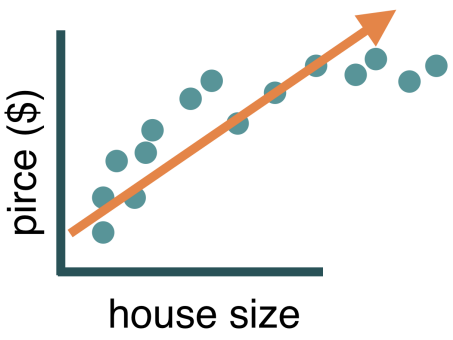


Figure 9: High Bias

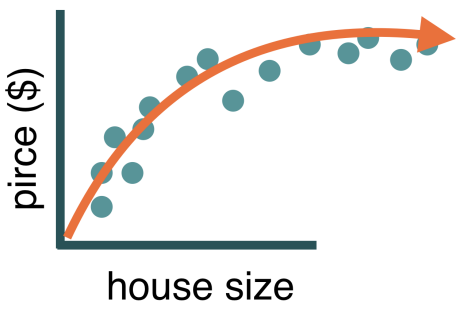


Figure 10: Just Right

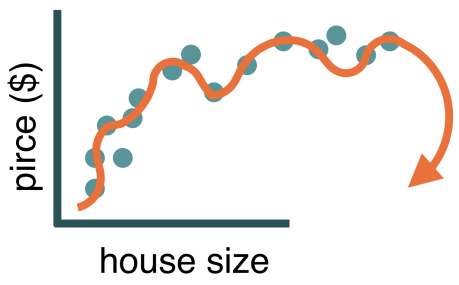


Figure 11: High Variance

Average over the distribution of the training data

Expected risk

$$\mathbb{E}_{\mathcal{D}}\left[R[h_{\mathcal{D}}(\boldsymbol{x})]\right] = \int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}$$

Namely, the randomness with respect to $\mathcal{D}$ is marginalized out.

Averaged prediction

$$\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) = \int_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) P(\mathcal{D}) d\mathcal{D}$$

Namely, if we have seen many training datasets, we predict with the average of the prediction functions learned on each training dataset.

Variance

We will subtract the averaged prediction from the averaged risk

$$
\begin{aligned}
\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\boldsymbol{x})] &= \int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D} \\
&= \int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) \\
&\qquad + \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D} \\
&= \underbrace{\int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}}_{\text{VARIANCE}} \\
&\qquad + \int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}
\end{aligned}
$$

Where does the cross-term go?

It is zero

$$\int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})][\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y] p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}$$

$$= \int_{\boldsymbol{x}} \int_{y} \left\{ \int_{\mathcal{D}} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})] P(\mathcal{D}) d\mathcal{D} \right\} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y] p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

$$= 0 \leftarrow \text{(the integral within the braces vanishes, by definition)}$$

Analyzing the variance

Understanding the variance

$$\int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}$$

$$= \int_{\boldsymbol{x}} \int_{y} \left(\int_{\mathcal{D}} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})]^2 \; P(\mathcal{D}) d\mathcal{D} \right) p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

For each $(\boldsymbol{x}, y)$ pair, we compute the squared difference of $h_{\mathcal{D}}(\boldsymbol{x})$ (the prediction with training dataset $\mathcal{D}$) and the averaged prediction $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$: the average (over all $(\boldsymbol{x}, y) \sim p$) variance of the prediction over $\mathcal{D}$.

How can we reduce the variance?

- Use a lot of data (ie, increase the size of $\mathcal{D}$)
- Use a simple $h(\cdot)$ so that $h_{\mathcal{D}}(\boldsymbol{x})$ does not vary much across different training datasets. An extreme example is $h(\boldsymbol{x}) = \text{const}$.

The remaining item

$$\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\boldsymbol{x})] = \int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}$$
$$+ \int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}$$

The integrand has no dependency on $\mathcal{D}$ anymore and simplifies to

$$\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

We will apply a similar add-and-subtract trick, by using an averaged target y (what we want to predict from $\boldsymbol{x}$):

$$\mathbb{E}_y[y|\boldsymbol{x}] = \int_y y p(y|\boldsymbol{x}) dy$$

Bias and noise

Decompose again

$$\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

$$= \int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_y[y|\boldsymbol{x}] + \mathbb{E}_y[y|\boldsymbol{x}] - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

$$= \underbrace{\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_y[y|\boldsymbol{x}]]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy}_{\text{BIAS}^2}$$

$$+ \underbrace{\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_y[y|\boldsymbol{x}] - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy}_{\text{NOISE}}$$

Where is the cross-term?

Take-home exercise: Show that it is zero

Analyzing the noise

How can we reduce noise?

$$\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_y[y|\boldsymbol{x}] - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy = \int_{\boldsymbol{x}} \left(\int_{y} [\mathbb{E}_y[y|\boldsymbol{x}] - y]^2 p(y|\boldsymbol{x}) dy \right) p(\boldsymbol{x}) d\boldsymbol{x}$$

There is nothing we can do. This quantity depends on $p(\boldsymbol{x}, y)$ only; choosing $h(\cdot)$ or the training dataset $\mathcal{D}$ will not affect it. Note that the integral inside the parentheses is the *variance* (noise) of the posterior distribution $p(y|\boldsymbol{x})$ at the given $\boldsymbol{x}$.

Figure 12: Somewhat difficult posterior

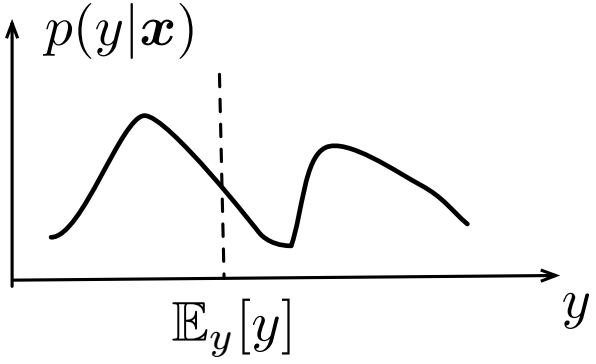
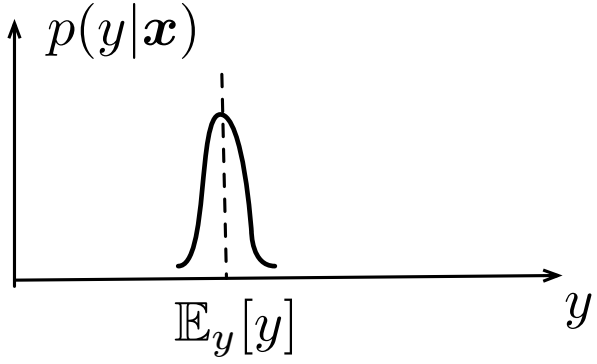


Figure 13: Somewhat easy posterior



Analyzing the bias term

Understanding the bias

$$\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{y}[y|\boldsymbol{x}]]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy$$

For each $(\boldsymbol{x}, y)$ pair, we compute the loss of our averaged prediction $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$ compared to the expected value of y given $\boldsymbol{x}$, which we compute as $\mathbb{E}_{y}[y|\boldsymbol{x}] = \int_{y} y p(y|\boldsymbol{x}) dy$. Then we take the average over all pairs $(\boldsymbol{x}, y) \sim p(\boldsymbol{x}, y)$.

How can we reduce the bias?

It can be reduced by using more complex models. We shall choose $h(\cdot)$ to be as flexible as possible: the better $h(\cdot)$ approximates $\mathbb{E}_{y}[y|\boldsymbol{x}]$, the smaller the bias. However, this will increase the VARIANCE term.

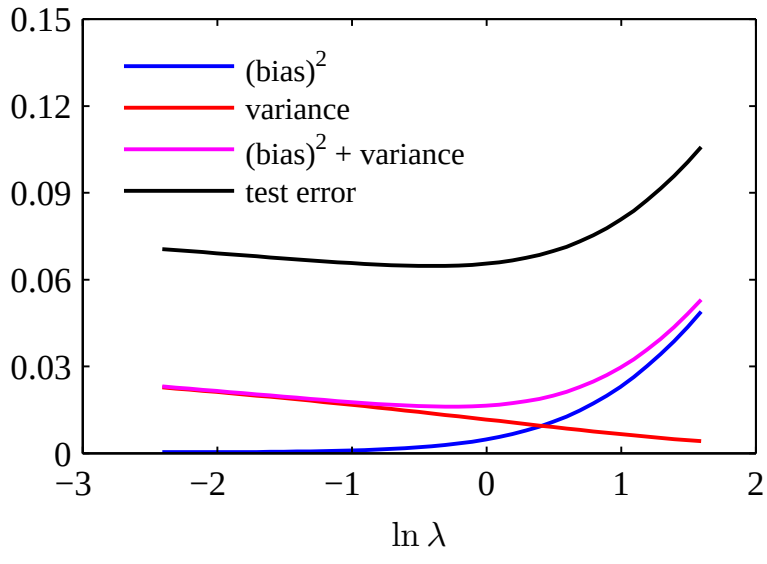
Bias/variance tradeoff

Error decomposes into 3 terms

$$\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\boldsymbol{x})] = \text{VARIANCE} + \text{BIAS}^2 + \text{NOISE}$$

where the first and the second term are inherently in conflict in terms of choosing what kind of $h(\boldsymbol{x})$ we should use (unless we have an infinite amount of data).

If we can compute all terms analytically, they will look like this



Summary of risk components

The average risk (with quadratic loss) can be decomposed as:

$$\mathbb{E}_{\mathcal{D}} R[h_{\mathcal{D}}(\boldsymbol{x})] = \underbrace{\int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}}_{\text{VARIANCE: error due to training dataset}}$$

$$+ \underbrace{\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{y}[y|\boldsymbol{x}]]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy}_{\text{BIAS}^2\text{: error due to the model approximation}}$$

$$+ \underbrace{\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{y}[y|\boldsymbol{x}] - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy}_{\text{NOISE: error due to randomness of } y}$$

Here we define: $h_{\mathcal{D}}(\boldsymbol{x})$ as the output of the model trained on $\mathcal{D}$, $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$ as the expectation of the model over all datasets $\mathcal{D}$, and $\mathbb{E}_{y}[y|\boldsymbol{x}]$ as the expected value of y.

Example: Why regularized linear regression could be helpful?

Model

$$h(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$$

Consider the best possible (linear) $h^*(\boldsymbol{x})$

$$\boldsymbol{w}^* = \text{argmin}_{\boldsymbol{w}} \int_{\boldsymbol{x}} [\mathbb{E}_y[y|\boldsymbol{x}] - \boldsymbol{w}^\top \boldsymbol{x}]^2 p(\boldsymbol{x}) d\boldsymbol{x}$$

Note that this linear model assumes the knowledge of joint distribution, thus, not achievable. Intuitively, it is the *best* linear model that can predict the data most accurately.

More refined decomposition of the bias

$$\int_{\boldsymbol{x}} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_y[y|\boldsymbol{x}]]^2 p(\boldsymbol{x}) d\boldsymbol{x} = \int_{\boldsymbol{x}} [h^*(\boldsymbol{x}) - \mathbb{E}_y[y|\boldsymbol{x}]]^2 p(\boldsymbol{x}) d\boldsymbol{x} + \int_{\boldsymbol{x}} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - h^*(\boldsymbol{x})]^2 p(\boldsymbol{x}) d\boldsymbol{x}$$

- Model bias: the price we pay for choosing linear functions to model data. This is the difference between the prediction of the best possible linear model and the actual target.
- Estimation bias: the difference between the optimal model and the estimated model.

Normally, the estimation bias is zero if we do not regularize.

Bias/variance tradeoff for regularized linear regression

We can only adjust estimation bias

$$\int_{\boldsymbol{x}} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}; \lambda) - h^*(\boldsymbol{x})]^2 p(\boldsymbol{x}) d\boldsymbol{x}$$

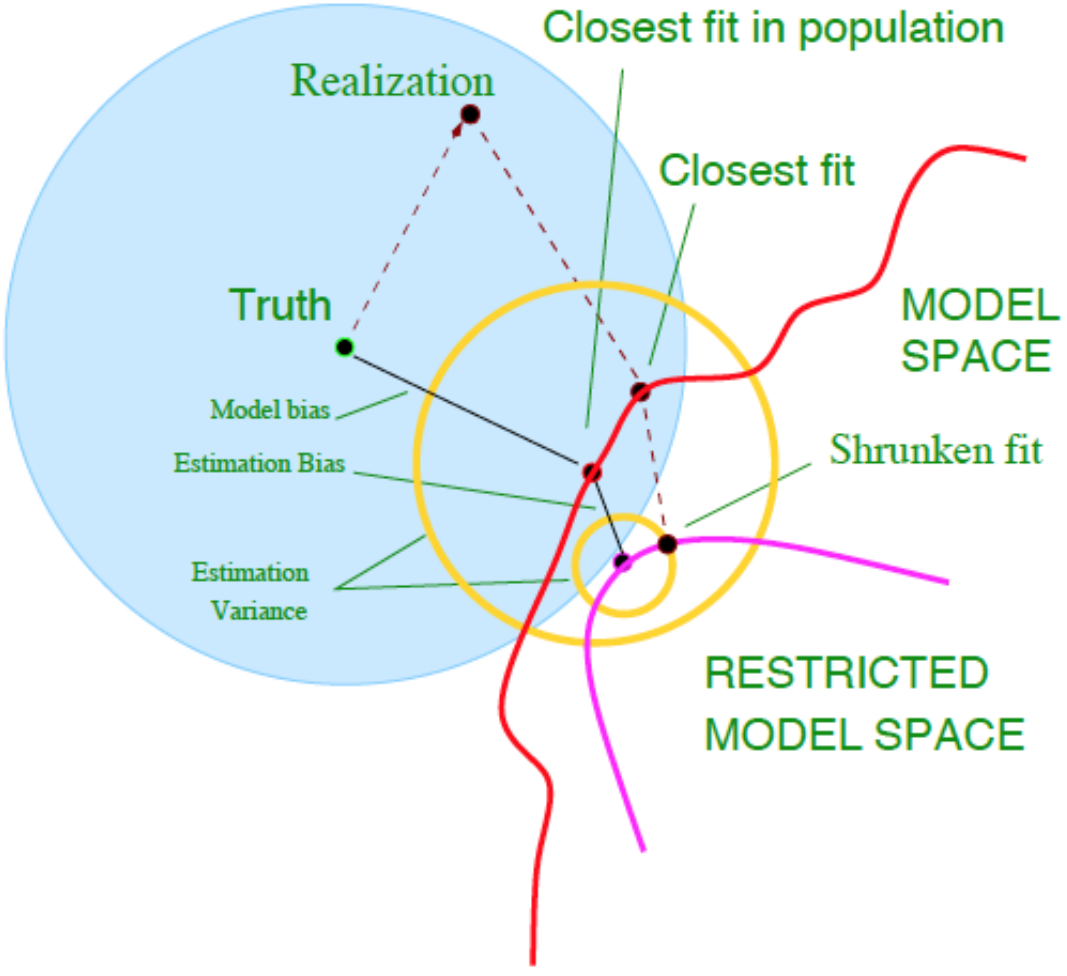
where $h(\boldsymbol{x}; \lambda)$ is the estimated model with regularized linear regression (parameterized with λ).

This term will not be zero anymore!

Thus, bias goes up.

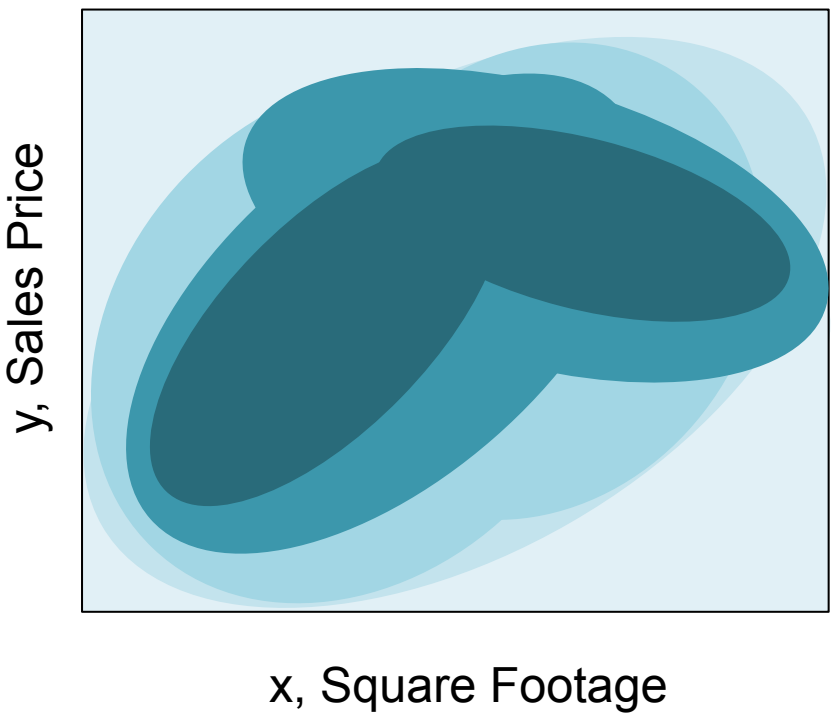
But, as long as this is balanced with a decrease in variance, we are willing to do so.

Visualizing the tradeoff



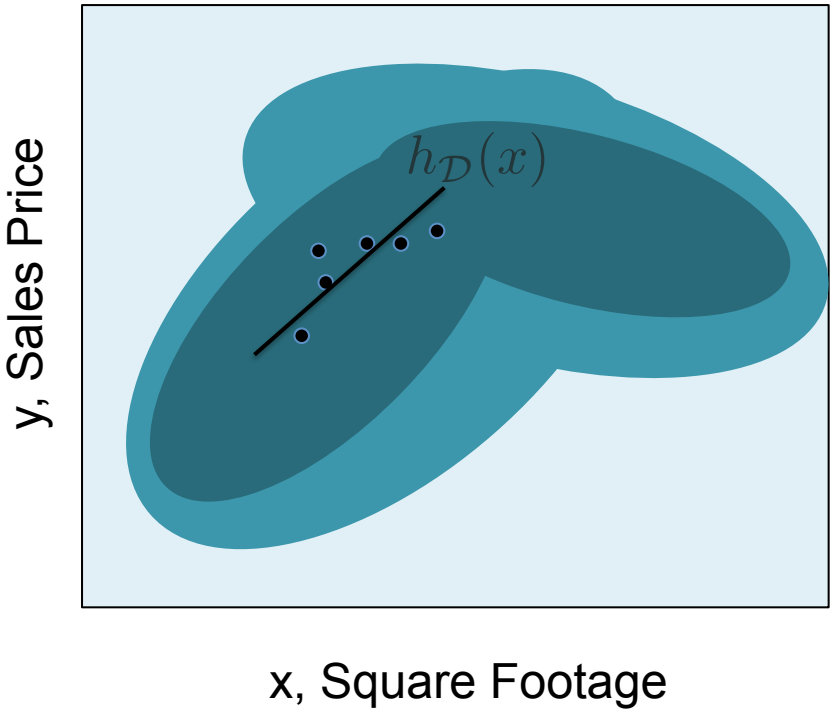
Bias-Variance Trade-off: Illustration

- Joint distribution of square footage x and house sales price y
- Darker color indicates higher probability regions



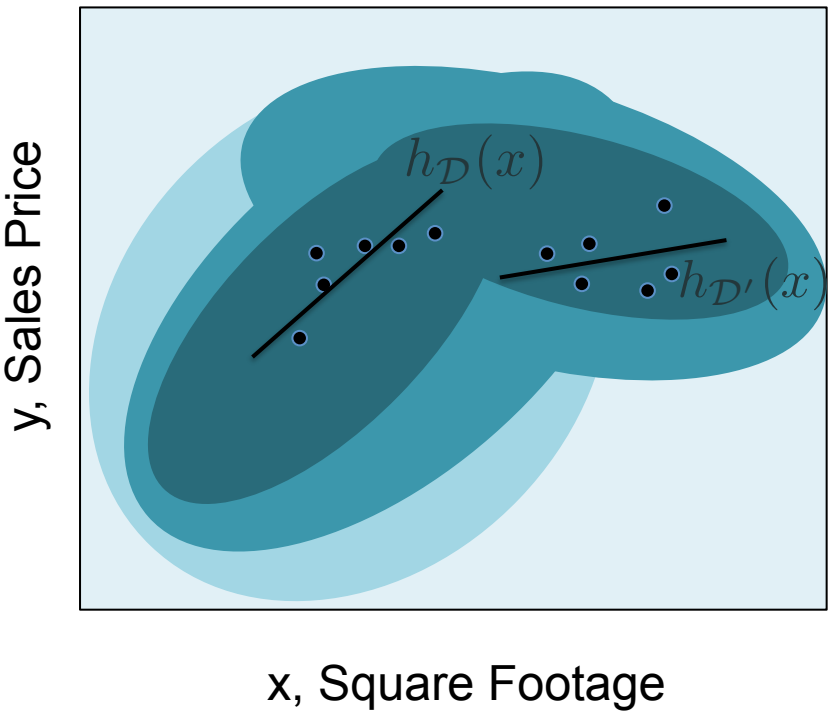
Bias-Variance Trade-off: Illustration

- We have access to dataset $\mathcal{D}$ sampled from the joint distribution
- Learn linear model $h_{\mathcal{D}}(x)$ based on $\mathcal{D}$



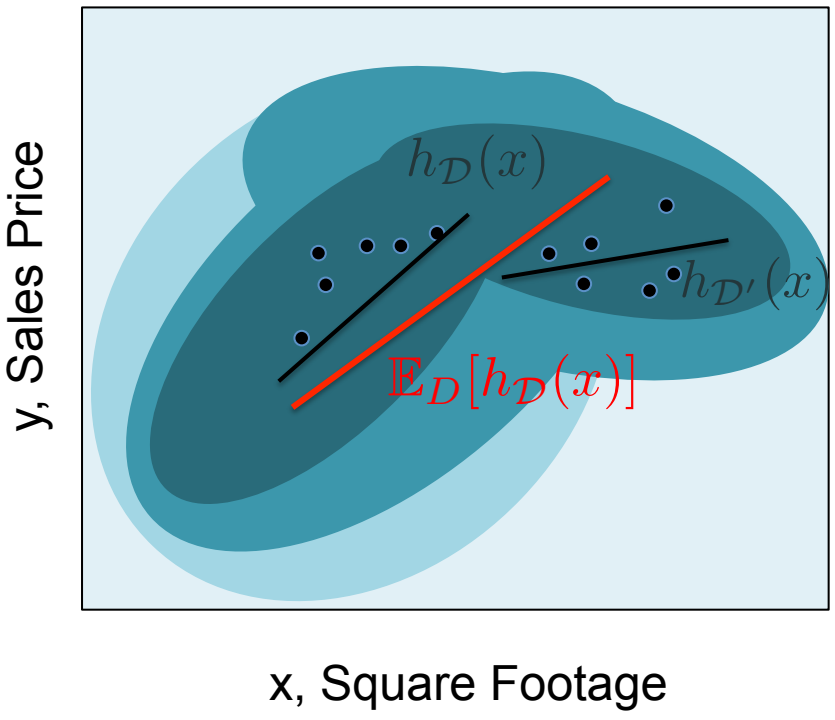
Bias-Variance Trade-off: Illustration

- Repeating this process for a different dataset $\mathcal{D}'$ yields a new linear model $h_{\mathcal{D}}(x)$
- Average of such models over infinitely many datasets sampled from the joint distribution is $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$



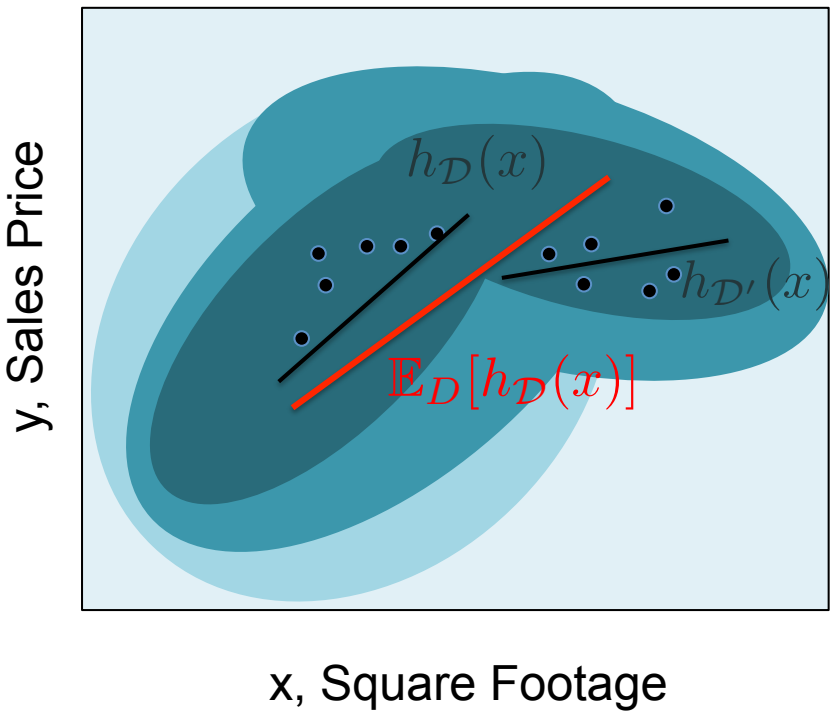
Bias-Variance Trade-off: Illustration

- Repeating this process for a different dataset $\mathcal{D}'$ yields a new linear model $h_{\mathcal{D}}(\boldsymbol{x})$
- Average of such models over infinitely many datasets sampled from the joint distribution is $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$



Bias-Variance Trade-off: Illustration

- Average of such models over infinitely many datasets sampled from the joint distribution is $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$
- Variance term captures how much individual models differ from the average $\underbrace{\int_{\mathcal{D}} \int_{\boldsymbol{x}} \int_{y} [h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy \; P(\mathcal{D}) d\mathcal{D}}_{\text{VARIANCE: error due to training dataset}}$

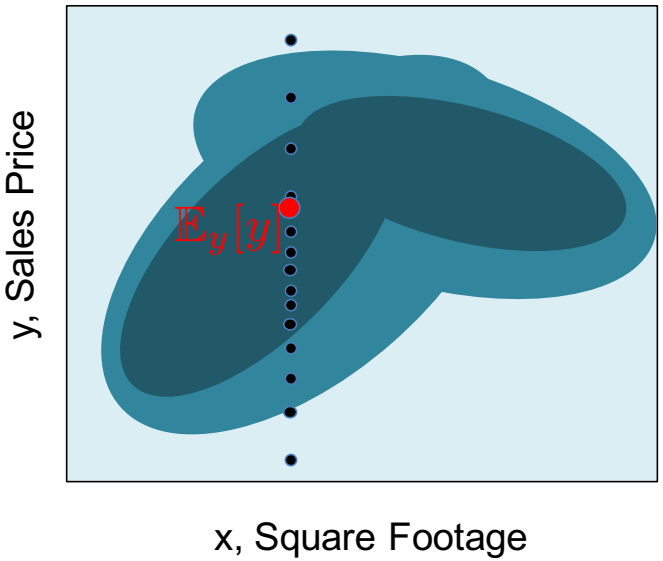


Bias-Variance Trade-off: Illustration

- For a given $\boldsymbol{x}$, we have a conditional distribution $p(y|\boldsymbol{x})$ of sales prices; the Bayesian optimal prediction of the label value for a given $\boldsymbol{x}$ is $\mathbb{E}_y[y|\boldsymbol{x}]$;
- The noise term measures the inherent variance in labels y

$$\underbrace{\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_y[y|\boldsymbol{x}] - y]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy}_{\text{NOISE: error due to randomness of } y}$$

- This term has nothing to do with our prediction $h_{\mathcal{D}}(\boldsymbol{x})$



Bias-Variance Trade-off: Illustration

- If our model class was rich enough (eg. a high-degree polynomial), then $\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x})$ should perfectly match $\mathbb{E}_y[y|\boldsymbol{x}]$
- Restricting to simpler models (eg. linear) results in a bias

$$\underbrace{\int_{\boldsymbol{x}} \int_{y} [\mathbb{E}_{\mathcal{D}} h_{\mathcal{D}}(\boldsymbol{x}) - \mathbb{E}_y[y|\boldsymbol{x}]]^2 p(\boldsymbol{x}, y) d\boldsymbol{x} dy}_{\text{BIAS}^2\text{: error due to the model approximation}}$$

